

A LEVEL COMPUTER SCIENCE NEA
PROJECT

DRS SYSTEM

BY

BRIJESH SAVJANI

Photo by Mudassir Ali on Unsplash

github.com/BrijeshSavjani
brijeshnsavjani@gmail.com

Contents

DEFINITIONS	3
ANALYSIS	3
THE PROBLEM	3
WHY USE A COMPUTER?	4
HOW WILL THE COMPUTER SOLVE THE PROBLEM?	4
WHO IS THIS AIMED AT?	4
EXISTING SOLUTIONS	4
HAWKEYE	4
HOTSPOT	5
ULTRA-EDGE/ REAL TIME SNICKOMETER (SNICKO).....	5
EVALUATION OF EXISTING SOLUTIONS	5
SIMILAR SOLUTIONS IN OTHER SPORTS	6
GOLF.....	6
BASEBALL.....	7
SURVEY OF WANTED FEATURES	7
WHO I'M ASKING?	7
THE QUESTIONS.....	7
THE RESPONSES	8
WHICH RESEARCHED FEATURES WILL BE INCORPORATED IN MINE?.....	9
ESSENTIAL FEATURES.....	10
LIMITATIONS OF MY PROGRAM	10
REQUIREMENTS	11
SOFTWARE.....	11
HARDWARE.....	11
SUCCESS CRITERIA.....	12
DESIGN	14
GUI DESIGN:	14
LBW BACKEND DESIGN:.....	14
ULTRA-EDGE PLAN.....	16
SPRINTS.....	17
DEVELOPMENT	19
SPRINT 1 (AV PROCESSING)	19
SPRINT 2 (QUADRATIC SOLVER).....	22
SPRINT 3 DETERMINE HEIGHT OF BALL	37
SPRINT 4 DETERMINE LATERAL MOVEMENT OF BALL	44
SPRINT 5 (OPENCV OBJECT DETECTION)	47
SPRINT 6 (EXTRACTING REAL WORLD CO-ORDINATES OF BALL)	53

SPRINT 7 (ASSEMBLING EVERYTHING TOGETHER)	60
TESTING & EVALUATIONS	63
SUCCESS CRITERIA	63
TESTING THE CODE AGAINST SUCCESS CRITERIA	65
SURVEY QUESTIONS	69
RESULTS OF SURVEY	69
IF I HAD MORE TIME AND WHAT THINGS I WOULD DO DIFFERENTLY	70

Definitions

This report will use subject specific terminology, so I felt that it was best to define it all for those who are not familiar with the game of cricket.

Cricket – A team-based bat and ball sport that involves two teams of 11 players each player. In this game bowlers (players who are deliver the ball to the batsman) try to get the batsman out and/or stop them from scoring points (called runs). A batsman can get out in 4 ways (Bowled, LBW, Caught and Stumped/Run Out).

Bowled – When the bowler bowls(delivers) the ball and the ball hits the stumps (3 sticks behind the batsman). If this happens then the batsman is out.

LBW – If the batsman hits the ball with their legs (instead of with the bat) and the ball would have gone on to hit the stumps then the batsman is out LBW.

DRS – DRS stands for Direct Review System and is a group of technologies (including Ball tracking, Ultra edge/Snicko, Hotspot and replays)

Edged - This is when the batsman hits the ball with edge of the bat. This means that they are not in control of where the ball goes and often ends up with them being caught.

Analysis

The problem

Picture this: you are playing a nice game of cricket in the garden and then bam! Something happens and everyone can't agree exactly what happened. Did the batsman edge the ball? Would the ball have hit the stumps? We've all been there, right? Well, this app aims to end those conflicts in the sport of cricket. It aims to give gardens and small pitches the exact same facilities as international level matches at a fraction of the cost. Yes, that's right I aim to recreate the core features of the DRS system at a fraction of the cost and allow it to run on mobile phones.

Why use a computer?

The alternative to using a program like mine to determine if a delivery has produced a wicket is to use a human. Since the players are disagreeing and have their own biases a neutral third party would need to be used (an umpire). However, for this an umpire would have to be hired and this would be very expensive and excessive. In this scenario the players could just ask my software for an unbiased and fair decision. Umpires are also prone to human error whereas technology is precise, consistent and error free.

How will the computer solve the problem?

I plan on developing a mobile app that would use the hardware of the mobile phone to record the ball. Then if need be, this footage can be analysed to determine if a wicket has fallen. This, much like the real systems, would require human input. However, this input will be indisputable data, such as "What colour is the ball?" or "Select the frame in which the ball leaves the hand".

The computer can then use this data to run simulations (in the case of LBWs) or to analyse inputted audio to determine if the batsman has lightly hit the ball (edged it) and then return its decision. These algorithms may not all be run on the local machine (mobile phone) due to processing power and language constraints. This means data may need to be sent to a web server.

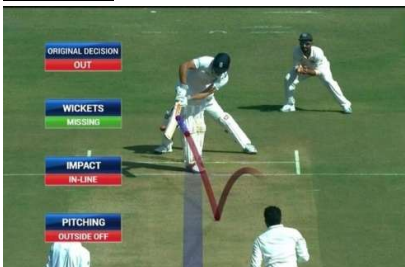
Who is this aimed at?

This is aimed at casual cricket players. This would especially appeal to cricketers who play mini games with each other that tend to be quite competitive because this would give a definitive answer to any appeals the bowler makes for a wicket.

This solution will fulfil the needs of those players by being fast & easy to set up so not much time is wasted. Reviews should also be easy to make and be processed quickly for the same reason. If that happens then it will fulfil the needs of the target market as it will be an objective, accurate and quick way to settle arguments.

Existing Solutions

Hawkeye



Hawkeye Innovations is an English Software company that developed the ball tracking software for international cricket. Their software has been calculated to be correct within a margin of 3.6mm (according to an independent review by the ICC).

Hawkeye has not released the exact method they use to predict the path of the ball (as it is proprietary) but we do

know that it uses 6 high speed (340 fps) cameras at different angles in the stadium. These cameras

follow the ball from the instant it leaves the bowler to the instant it hits the batsman in the leg. This can then be used to calculate speed, velocity, displacement, spin, lateral movement etc. As you may be able to see hawk eye differentiates the predicted path of the ball and the path of the ball they tracked.



Hawkey also does other cricket related things for broadcasters such as tracking where a bowler lands the ball on a pitch overtime or wagon-wheels that show where the batsman has hit balls.

Hotspot



Hotspot is a piece of software that was developed by Nicholas Bion that aims to detect edges. His software utilizes 4 infrared cameras that have been placed at different angles in the stadium. Bion realized that friction is created whenever the bat hits the ball, and that friction creates heat. This gave him the idea to set up infrared cameras to detect the heat produced. He found when a batsman has edged/hit the ball a white 'hotspot' is created.

However, Hotspot has recently been embroiled in controversy for not picking up very clear edges. This is because the protective coatings on modern bats are very smooth, so friction is not always detectable.

Ultra-edge/ Real time Snickometer (Snicko)



Snicko and Ultra-edge are two competing software's that both use the same theory and hardware. Snicko came first and was developed by Alan Plaskett whereas Ultra-edge is a copy-cat software created by Hawk eye.

Snicko and Ultra-edge use a mic hidden in the stumps (the three sticks behind the batsman) to record audio. The footage and audio are slowed down, then the waveform of the audio is overlayed. If there is a spike in the wave as the ball passes the bat, then the batsman must have edged it to generate that sound.

Evaluation of existing solutions

Hawkey is a very good system as it is very accurate (within 3.6mm of where the ball ends up in tests). Furthermore, the graphics produced to show the result is informative and functional as it shows what the prediction is and where the software thinks the ball is going to go. However, the

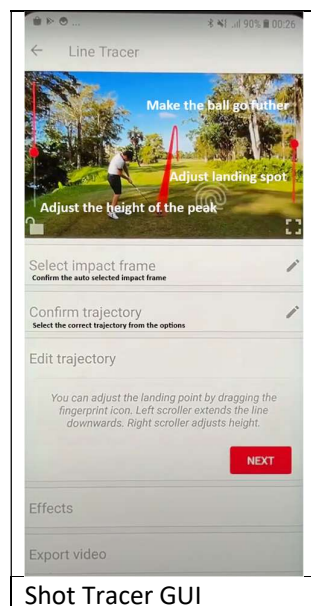
system itself is quite costly and 6 camera angles at 340FPS is a bit excessive for a small pitch/ garden so this will need to be simplified.

Hotspot is a handy technology to have but it's inaccuracies with new bats makes it controversial. Furthermore, high-res infrared cameras are expensive. The ones used in professional stadiums cost 6000 GBP per day each. A low-res IR camera would probably not work as it would probably not be able to clearly detect the amount of heat the friction has produced (as it is very little).

Ultra-edge / Snicko is a great inclusion as it is simple to develop, does not require expensive hardware and just works.

Similar Solutions in other sports

Golf



Professional and amateurs in golf usually use a tier of ball tracking software that has been released by a Company called Shot Tracer. They do have a mobile app for amateurs that allows them to measure their shot length, power, and direction. As well as to form a trace of the ball path. However, they also recommend that you use their specialized devices for tracking.

I will be analysing their app for my project to see how it works and what features to include in my project. To track a golf ball you can either upload a video or record a new one. The app first analyses the sound from the clip to determine the point of impact. It then traces the shot of the ball using its proprietary technology. It gives the user multiple choices for the trajectory but selects the one it thinks is most likely. It then prompts the user to select where the ball landed. It also uses their proprietary system to determine when the ball lands and auto trim the video.

Whilst I did like the simplicity of the UI, I thought that the proprietary tracking system was very weak as the user had to manually select and correct every parameter such as the distance the ball traveled, the height of the ball and the landing spot. I believe this is because it is a single camera system, so I will make my system dual camera to avoid this. I loved the automatic impact detection as I thought the idea of using sound was incredibly smart however this won't work in my system as in cricket the ball doesn't make enough sound when it hits the pad. Furthermore, even if it did the sounds from bowler's celebrations/batsman's frustration would probably confuse the system.

In conclusion from the 'Shot Tracer' app I will be taking the simple UI and ball tracking. However, I will be modifying the ball tracking to be more precise and not need as much user input as this is time consuming and subjective.



The Rapsodo PITCHING 2.0 app

Baseball

In baseball pitchers throw a variety of balls (much like cricket) and often want feedback instantly. So a company called Rapsodo developed a device that can be linked to your phone and show you the path of the ball, it's velocity and it's spin among other things. I will be analysing their PITCHING 2.0 system for my project.

The Rapsodo device is a small box that contains a series of cameras and a RADAR. This box then syncs up with a tablet app. It uses its proprietary software along with the data it collects to analyse the data and then almost instantly report its findings.

I really like the almost instant feedback from the Rapsodo solution so I would really like to try and emulate this in my project. I also liked the idea of the device syncing to a more powerful device for processing so this is something I definitely would like my app to do as cheap phones are not known for their processing power. I also liked how it stored the data from previous pitches for further analysis so I would like to add this on if I get more time. However, I did not like the \$1500/year price tag. I felt like this excessively priced out of the range of most consumers, so I won't be using this.

Survey of wanted features

Who I'm asking?

As I previously mentioned in my 'Who is this aimed at' section my target users are people who play mini games in the netes/backyard against each other. For my survey, I decided to ask the people in my club for their opinion as they are all the kind of people who would use this product after it has been developed.

The Questions

I decided to ask a broad range of questions to help me analyse the results afterwards to help determine what features need to be added in. I made a form in Microsoft Forms with the following questions:

My first question was: "What methods of dismissal are allowed in nets/backyard games?" I asked this first question to contextualize the responses to the next question (which form of dismissal is most contentious). As, if a form of dismissal isn't allowed in their game, then it obviously can't be the most contentious. I gave the options of: Bowled, Caught, LBW and Edged (Auto/Electric wicketkeeper). NOTE: Auto/Electric wicket keeper means that if the batsman is automatically out if the ball gets edged back onto the wall/net behind the batter.

My next question was: “What mode of dismissal is most contested?” and the options were the same as the first question. I asked this question to help gauge which forms of dismissals to review and which ones could be left out of my system. I expect LBW and Edged to be the most contentious as these are the most subjective.

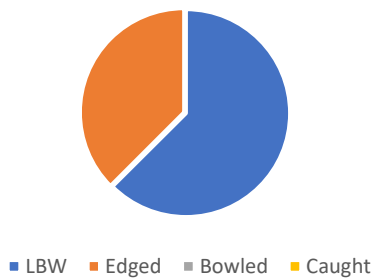
The third question was: “Would you use a DRS system in the nets/backyard games to remove controversy when a batsman doesn't walk?”. The options were: “Yes”, “No” and “Maybe”. This was to gauge if people wanted a system like this or not.

The penultimate question was: “Would analysis of your bowling (such as pitch maps and amount of movement) help you improve your bowling?”. The options were either yes or no. I asked this question after I found that existing solutions like PITCHER 2.0 had this feature.

The last question I asked was: “If you answered yes to either of the previous two questions, how long would you be happy to spend setting one up?”. I then gave 5-minute ranges as options from 0-5 mins all the way to 20+ mins. This was to gauge how long I should aim my set up time as. Because of the additional complexity of adding another camera, microphone and syncing it all up I knew it was going to take a while. But I wanted to know roughly how many minutes I’d need to cut this down by.

The Responses

Most contested form of dismissal (all)



All in all, I got about 15-20 responses. I made the following charts to help understand and interpret the collected data.

This first Pi-Chart shows that the most contested form of dismissal by a large margin is LBW. Then in a large minority is Edged and caught behind. Bowled and caught didn’t get any votes.

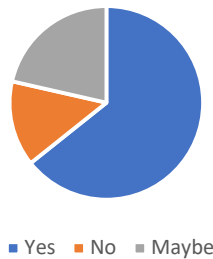
Most Contested form of dismissal w/
Auto-keeper



When analysing the data from the first question I realized that not a lot of people played with the auto keeper rule so I decided to analyse the responses of the people who did to see if LBW was still the most contested form of dismissal. Edged behind had a small majority over LBW but because of the small amount of data points it’s almost 50/50.

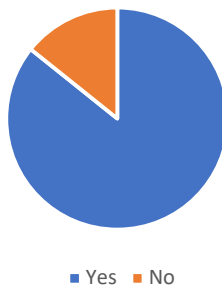
This led me to the conclusion that both are almost equally contested.

Would you use a DRS system?



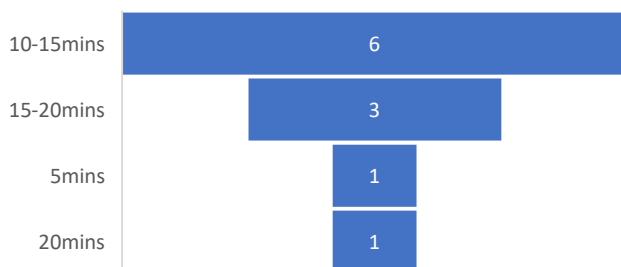
I then analysed if people actually would want a system like the one, I was going to develop. The majority of people said either yes or maybe. This validated the need of a software like this to exist.

Would bowling analysis help you?



I then checked if a “Bowling Analysis” feature like the one found in PITCHER 2.0 would help people improve their bowling. A vast majority of people said it would.

How long would you be happy to set up for?



As you can see most respondents (who said yes to the system) said that they would be happy to spend 10-15 minutes. So, this will be my minimum initial setup time target.

Which researched features will be incorporated in mine?

I plan to include a version of the ball tracking found in Hawkeye. This is because most

respondents said this was the most contentious form of dismissal. However, I will use two normal cameras instead of 6 cameras. This means it will be less accurate and have less features, but my software doesn't need all the features. But it needs at least two as we can see from the Golf 'Shot Tracer' app that one camera is too inaccurate and requires too much user input. This is because 1 camera can't tell depth. It will only need the ball tracking and path prediction from Hawkeye. I will most likely use two mobile phones instead of two cameras. This is because it is easier to sync two phones to record at the same time. I also need a display to display the result and for the ball analysis (that was very popular in my survey).

As edged behind is the most contentious dismissal when it's allowed, I have to include a system to accommodate reviews, but I will not include Hotspot because of the limitations of the software and cost. It will also make the system harder to set up as more things will need to be placed and synced. I believe that Snicko will be enough for edge detection. I will include all Snicko in my software. This is by far the easiest one to include and I believe it will be quite useful to detect edges. I will have a mic hidden behind the stumps in the ground and I will connect it to one of the phones (the side on one) using an AUX cable.

As the majority of people said they would use and would improve by using a ball analyser (like the one found in PITCHER 2.0), I felt I had to include it if I get time. For the UI of the analyser/ app I will make a simple, modern UI like the one found in 'Shot Tracer'.

In conclusion my software will be able to check all forms of dismissal using only two mobile phones. I am aware that this will be less accurate than the actual systems however my target market is not the highest level of cricket so more leeway should be afforded (in my opinion).

Essential features

The ball tracking and prediction section is an essential feature of my program. This will probably be the most used feature because the outcome is the least clear to the players (as my survey results indicated). However, this will be the most difficult feature so I will use decomposition to break this down into smaller features to make this less daunting and easier to develop.

Another essential feature is Review Management. This will just track how much reviews each team has done and restrict them to a fixed amount per game. This will force the players to think about reviewing a ball to see if there is a wicket. This is because if this feature was not in place, then every ball would be reviewed, and this would waste a lot of time in the game. However, each ball will still get automatically analysed so the bowler can use the data.

Edge detection will also be an essential feature of my program (the surveyees also asked for this). This is because edges are not always clear, and this would be a major source of arguments between the players if there is no 3rd party officiator. Edge detection is also required for LBW reviews as it will need to be checked if the ball hit the bat before the batsman leg or not.

Limitations of my program

As previously mentioned, my LBW reviews will be a lot less accurate than 3.6mm due to only having two angles and using normal speed cameras. I would be aiming for an accuracy of about 1.8cm (half the width of a ball). To compensate for this the Reviewing team will not lose their review if the ball is

within 1.8cm of the stumps. Furthermore, my predictions are likely to be less accurate as I will not be accounting for wind resistance. This is because this is likely to have a negligible on the result and will be super complicated to include. Furthermore, I don't know exactly how Hawkeyes' solution works so I am using my limited knowledge of mechanics and whatever I can research. However, this will probably still end up being a simple approximation.

The edge detection will also be slightly limited as I will not be using the same expensive super sensitive microphone. However, I believe it will still pick up most edges so it should still work. It will also be further limited by the lack of super slow-mo video (due to the normal FPS of the phone cameras) this means it will not be as clear cut to detect the edge.

My program should solve the problem it set out to solve completely. That is if everything goes to plan.

Requirements

Software

FFMPEG (Video & Audio Processing)
Python (Quadratic Solver & Open CV & scikit-learn/TensorFlow)
React Native (Making the App)
OpenCV (Identifying & Tracking the ball)
Android (To run the app)
Potentially TensorFlow/ SciKitLearn (For coefficient of restitution prediction)

Hardware

2X Mobile phone
AUX microphone
Webserver (Can just be local computer)
8GB RAM (scikit-learn/TensorFlow)
Decent enough CPU (scikit-learn/TensorFlow)

Wi-Fi booster/ Wi-Fi hotspot (so the phones can connect to the internet)

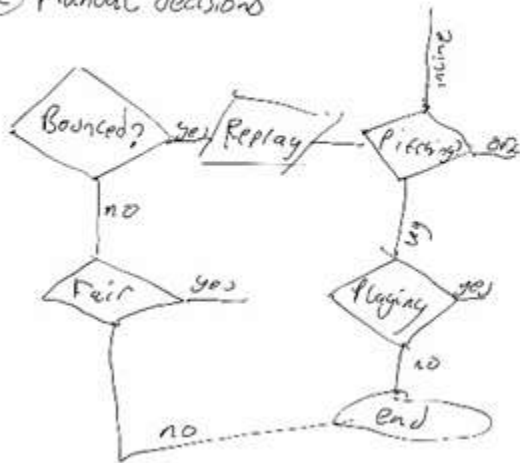
Success Criteria

#Ref	Criteria	How it improves the project/engagement?
1	Processing Time	
1.1	Be able to set up all cameras and microphones in less then 10-15 mins	Will ensure that the users don't spend too long setting it up to prevent them getting frustrated and not using it
1.2	Be able to quickly move all electrical equipment in less than a minute (with 2 ppl).	Will prevent the electrical items from breaking from water damage if it starts to rain quickly. At least 2 ppl can be used (batsman and bowler)
1.3	Process a LBW review with edge detection first in under two mins	Ensures that there is not a large break in play after every review. This time can simply be used to get a drink or for continued practice till the result comes back.
2	Have a reasonable degree of accuracy	
2.1	Be accurate to +- 1.8cm in LBW appeals	Will ensure that people continue to use it. If it was less accurate then this (half the balls diameter) then people would not use it.
2.2	Be able to detect edges	This was one of the most contentious form of dismissals so it's crucial to be able to detect them.
3	GUI	
3.1	Manage and record the amount of LBW reviews done by each time	Prevents players from reviewing any slight chance that will waste peoples time and eventually cause people to stop using it
3.2	Have GUI indicators to show if the cameras, mic and server are all connected	Will allow for quicker and easier set up as well as diagnostics.
3.3	Have buttons to allow the user to be able to select the correct from of review	Allow people to easily review dismissals using the right mode to see if people are out.

3.4	Display the waveform and slow-mo videos for edge detection	Help umpires/ the players decide if there is an edge very easily and quickly.
3.5	Have a setting menu for things like: Ball colour, Stump colour	Will allow the user to use the app on multiple different pitches, with different balls
4	LBW	
4.1	Identify the ball and stumps in a video automatically	Will speed up the time taken per review to prevent the user from getting frustrated and limits manual inputs.
4.2	Detect the path of the ball	The previous path of the ball is instrumental in determining if the ball will go on to hit the stumps or not in a LBW appeal
4.3	Predict the height of the ball	Important in deciding if the ball will hit the stumps or not as it will tell us if it goes over the stumps
4.3.1	<i>Calculate the vertical velocity of the ball</i>	Needed in conjunction with the Coefficient of Restitution to predict the height if its' going to bounce
4.3.2	<i>Predict the Coefficient of Restitution</i>	Needed in conjunction with the initial velocity to predict the height if its' going to bounce
4.4	Predict the horizontal movement of the ball	Needed to determine if the ball will be inline with the stumps or not.

① Review called

② Manual decisions



③ Isolate ball with filters

④ Detect balls co-ordinates and send to quadratic solver for height prediction.

⑤ if $\min(\text{heights}) \leq 0$

Initial Velocity

$r = \text{Coefficient Of Restitution}$

Final Velocity = Initial Velocity $\times r$

Max Height = $\left(\frac{\frac{1}{2} \text{Final Velocity}^2}{9.81} \right)$

New Equation = new Eq (motion)

Final Height = GA, Final Height

$\frac{1}{2} m v^2 = m g h$

$\frac{1}{2} v^2 = g h$

$\frac{1}{2} v^2 = g h$

$\frac{1}{2} v^2 = g h$

$\frac{1}{2} v^2 = g h$

In white box in case of multiple bounces

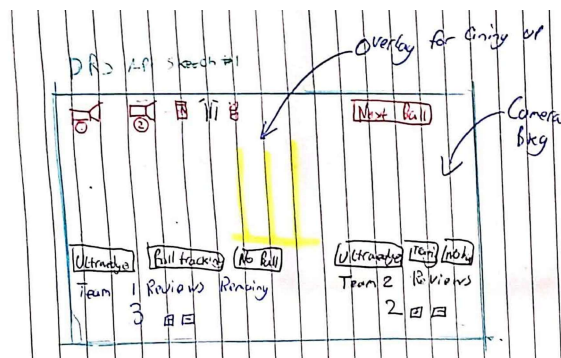
⑥ Work out lateral movement (Linear)

⑦ Compare height and horizontal position of ball. Return is hitting.

Design

GUI design:

The image at the bottom is my GUI design. I have designed this to have all the essential features such as: A counter for reviews left from each team, ability to call for all 3 types of reviews, status indicators for connected inputs (CAM1, CAM2, CAM3 (No Ball), MIC and Server). The background is the feed from the camera overlayed with an image which indicates where the app thinks the stumps are. This is for calibration and diagnostic purposes



Hardware Placement

The hardware will be placed in the following places. This is so that there are always two separate views on the ball so it's position in 3D space can be determined. The mic is also near the stumps so it can hear any edges. CAM 2 must be a wide angle lens. I haven't included CAM 3 on the diagram as it is not essential and will rather be an add-on if there is time to add it at the end.

LBW Backend design:

Some data must be manually inputted. The inputted data will be processed as the flow chart on the left indicates. All inputted data is objective (e.g., has the ball bounced) so there is no room for disagreements between the two teams. This is to reduce programming complexity. A replay will be played whilst the decisions are being inputted in case any information has been forgotten.

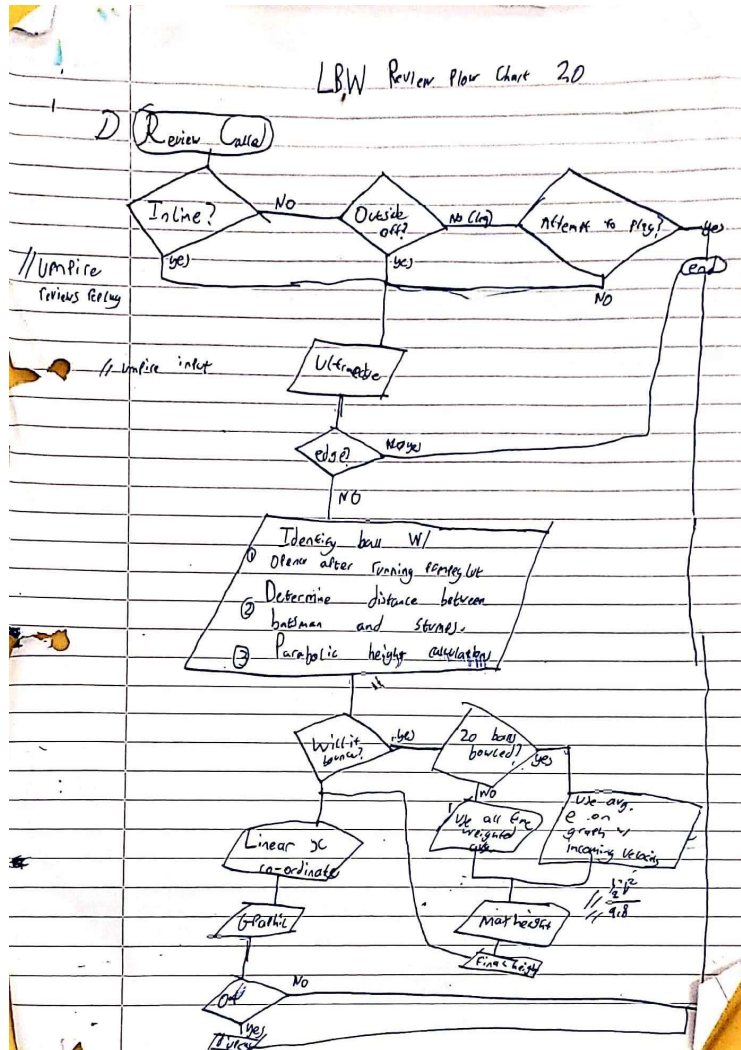
Next the program will crop and try to filter for the ball using FFMPEG filters and LUT transformations. This should make it easier for the computer vision software (OpenCV) to detect the ball. The cropping should also help reduce false positives that are out of the bounds of the pitch.

After the Computer Vision software has detected the co-ordinates of the ball, it should transform these into real time co-ordinates from known lengths such as height of the stumps or pitch length. These co-ordinates should then be passed onto a Quadratic Equation Creator (For Height) and a linear equation creator (for lateral movement).

The pseudo code on the side is a plan for what would happen if the ball was predicted to bounce. This is because if the ball bounces, then a new quadratic equation will be needed. The Coefficient of Restitution is how a measure of how much Kinetic Energy (and therefore velocity) an object keeps after a collision. This will be predicted by a Machine Learning algorithm such as K Nearest Neighbour.

The final step is to compare the predicted height and lateral position with that of the stumps. It will return if it's hitting or not. The result will then be presented to the user.

As a flow chart:

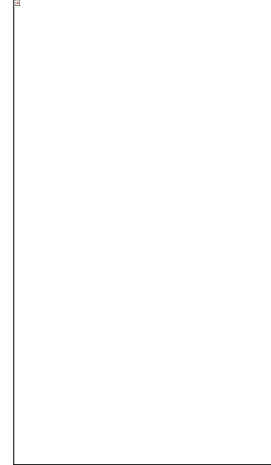
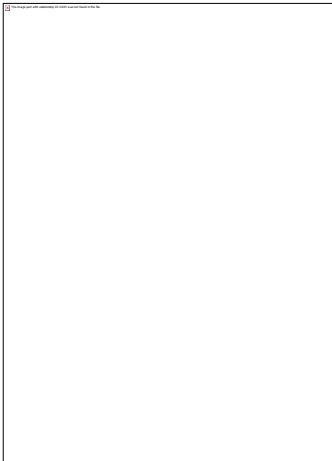


Ultra-edge plan



The video will be taken in from the camera and mic and slowed down to a much slower speed. The video will then have its audio split and removed. I will then generate and overlay the waveform of the audio onto the graphic on the side using FFMPEG. This will then be overlayed back onto the video. The video will then be displayed onto a front on view as well to give the most angles and sources to decide. Unfortunately, the software will not be able to decide if the batsman has edged it or not as it would be very hard for a computer system to distinguish between noise and a genuine edge.

Testing data



Sprints

Sprint #	Function								
1	AV Processing (Do the basic video and audio editing in FFMPEG). Sprint Needs To: <table> <tr> <td>1</td><td>Slow down videos and extract sound</td></tr> <tr> <td>2</td><td>Transform into blue wave form</td></tr> <tr> <td>3</td><td>Layer onto Ultra-edge logo</td></tr> <tr> <td>4</td><td>Slow down front and side angles of video.</td></tr> </table>	1	Slow down videos and extract sound	2	Transform into blue wave form	3	Layer onto Ultra-edge logo	4	Slow down front and side angles of video.
1	Slow down videos and extract sound								
2	Transform into blue wave form								
3	Layer onto Ultra-edge logo								
4	Slow down front and side angles of video.								
2	Quadratic Solver (Solve 3 sets of quadratics simultaneously to work out coefficients) Sprint Needs To: <table> <tr> <td>1</td><td>Take in 3 sets of X, Y, Z Co-ordinates</td></tr> <tr> <td>2</td><td>Calculate and return the respective values of A, B,C of the quadratic</td></tr> <tr> <td>3</td><td>Be able to detect if ball is going to bounce (if $y \leq 0$)</td></tr> </table>	1	Take in 3 sets of X, Y, Z Co-ordinates	2	Calculate and return the respective values of A, B,C of the quadratic	3	Be able to detect if ball is going to bounce (if $y \leq 0$)		
1	Take in 3 sets of X, Y, Z Co-ordinates								
2	Calculate and return the respective values of A, B,C of the quadratic								
3	Be able to detect if ball is going to bounce (if $y \leq 0$)								
3	Determine Height of ball (Only if ball bounces) Sprint Needs To: <table> <tr> <td>1</td><td>Estimate and Calculate/Retrieve stored Co-efficient of restitution</td></tr> <tr> <td>2</td><td>Use Coefficient of Restitution to calculate maximum height</td></tr> <tr> <td>3</td><td>Use this maximum height in conjunction with the bouncing point to work out final parabolic trajectory</td></tr> <tr> <td>4</td><td>Work out final vertical position</td></tr> </table>	1	Estimate and Calculate/Retrieve stored Co-efficient of restitution	2	Use Coefficient of Restitution to calculate maximum height	3	Use this maximum height in conjunction with the bouncing point to work out final parabolic trajectory	4	Work out final vertical position
1	Estimate and Calculate/Retrieve stored Co-efficient of restitution								
2	Use Coefficient of Restitution to calculate maximum height								
3	Use this maximum height in conjunction with the bouncing point to work out final parabolic trajectory								
4	Work out final vertical position								
4	Calculate Final Horizontal Deviation of Ball Sprint Needs To: <table> <tr> <td>1</td><td>Calculate Movement before bounce (Linear as through air)</td></tr> </table>	1	Calculate Movement before bounce (Linear as through air)						
1	Calculate Movement before bounce (Linear as through air)								

	2	Calculate Movement after bounce (Re-analyse footage after parabola hits root)										
	3	Return Final Position of ball										
5	OpenCV Object Detection (Detect Stumps and ball) Sprint needs to: <table><tr><td>1</td><td>Take in footage and split into frames</td></tr><tr><td>2</td><td>Apply any filters or transformations necessary for Computer Vision (e.g., RGB -> HSV)</td></tr><tr><td>3</td><td>Identify Ball through colour thresholding</td></tr><tr><td>4</td><td>Identify Stumps through colour thresholding</td></tr><tr><td>5</td><td>Contour two objects and export</td></tr></table>		1	Take in footage and split into frames	2	Apply any filters or transformations necessary for Computer Vision (e.g., RGB -> HSV)	3	Identify Ball through colour thresholding	4	Identify Stumps through colour thresholding	5	Contour two objects and export
1	Take in footage and split into frames											
2	Apply any filters or transformations necessary for Computer Vision (e.g., RGB -> HSV)											
3	Identify Ball through colour thresholding											
4	Identify Stumps through colour thresholding											
5	Contour two objects and export											
6	Extract real world co-ordinates of ball Sprint needs to: <table><tr><td>1</td><td>Identify and mark out point of reference</td></tr><tr><td>2</td><td>Determine ideal camera locations for said point of reference</td></tr><tr><td>3</td><td>Work out Pixels per Metre</td></tr><tr><td>4</td><td>Return X, Y & Z co-ordinates</td></tr></table>		1	Identify and mark out point of reference	2	Determine ideal camera locations for said point of reference	3	Work out Pixels per Metre	4	Return X, Y & Z co-ordinates		
1	Identify and mark out point of reference											
2	Determine ideal camera locations for said point of reference											
3	Work out Pixels per Metre											
4	Return X, Y & Z co-ordinates											
7	Work out position of final position ball in relation to stumps Sprint Needs To: <table><tr><td>1</td><td>Take inputs from functions and programs (created in previous sprints)</td></tr><tr><td>2</td><td>Determine/ask input for pitching and impact location</td></tr><tr><td>3</td><td>Plug in bounds of stumps to see if there's a collision. Return results</td></tr></table>		1	Take inputs from functions and programs (created in previous sprints)	2	Determine/ask input for pitching and impact location	3	Plug in bounds of stumps to see if there's a collision. Return results				
1	Take inputs from functions and programs (created in previous sprints)											
2	Determine/ask input for pitching and impact location											
3	Plug in bounds of stumps to see if there's a collision. Return results											
8	GUI Must Contain: <table><tr><td>1</td><td>Review tracker for both sides</td></tr><tr><td>2</td><td>Indicators of connected cameras</td></tr><tr><td>3</td><td>Buttons to call for review</td></tr><tr><td>4</td><td>Camera as background to app</td></tr><tr><td>5</td><td>Overlays to help align and calibrate pitch position</td></tr></table>		1	Review tracker for both sides	2	Indicators of connected cameras	3	Buttons to call for review	4	Camera as background to app	5	Overlays to help align and calibrate pitch position
1	Review tracker for both sides											
2	Indicators of connected cameras											
3	Buttons to call for review											
4	Camera as background to app											
5	Overlays to help align and calibrate pitch position											
9	Connect all devices to front end Devices: <table><tr><td>1</td><td>Camera 1</td></tr><tr><td>2</td><td>Camera 2</td></tr><tr><td>3</td><td>(Stump)mic</td></tr></table>		1	Camera 1	2	Camera 2	3	(Stump)mic				
1	Camera 1											
2	Camera 2											
3	(Stump)mic											
10	Connect backend to front end											

Development

Sprint 1 (AV Processing)

According to my prior planning this sprint must:

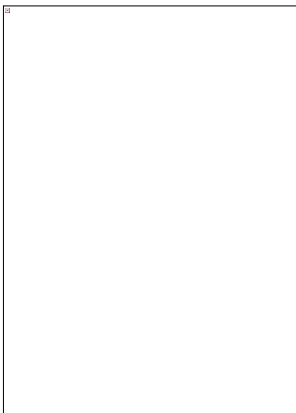
1	Slow down videos and extract sound
2	Transform into blue wave form
3	Layer onto Ultra-edge logo
4	Slow down front and side angles of video.

I reached several ways to accomplish those goals and I decided that a CLI program called FFMPEG was the best solution for this. FFMPEG is an opensource project that contains lots of libraries and programs to handle multimedia files. I decided this as it would be a lot quicker and simpler then developing the tools myself.

As this will be run in the backend of the program it doesn't need a UI but to make it more usable, I have overlayed the sound wave upon a white background to make it easier to read. I also amplified the sound so it's easier to spot a spike.

Input	Outputs
Front camera (Video – CAM1)	Ulatraedge.avi
Side camera (Video – CAM2)	front_out.avi
Ultra edge (white bkg) graphic	Ssde_out.avi

Testing Data:



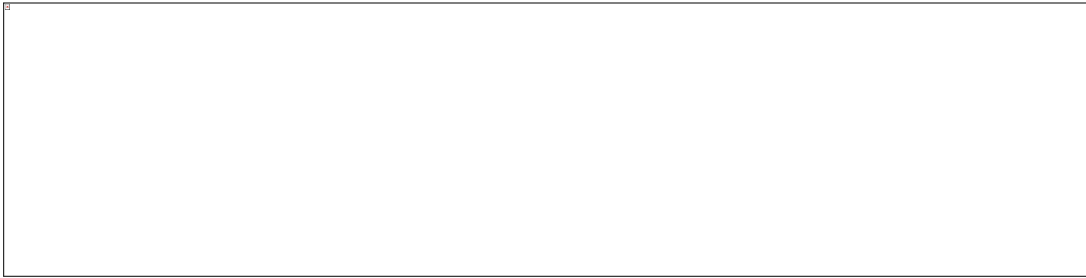
I used the clip to the left as my testing data as it shows an edge so we can use It to fine tune the amount of sound boosting needed to accurately detect an edge without detecting a phantom sound. I decided not to record a side angle as it is not required for testing as the processing is exactly the same.

Algorithmic Plan:



Slowing down the video & extracting sound

Extracting Sound



Firstly, the sound is extracted from the main video then it is slowed down to half the speed and doubled in volume. This is outputted into a file called output.mp3. Then output.mp3 is transformed into a waveform video with a size of 112*208px (The size of the image) and with a colour of #FFFF00(yellow). This is because the waveform will have a black background and we want blue lines on a white background. So, when we inverse colours, we get the desired output. Then all that is left is to overlay onto the Ultra-edge template image.

Video



The video will be recorded into two separate feeds (one from the front and the other from the side camera). They are both independently slowed to half-speed (same as the audio). This is so they all match up when layered together.

Deleting temp files

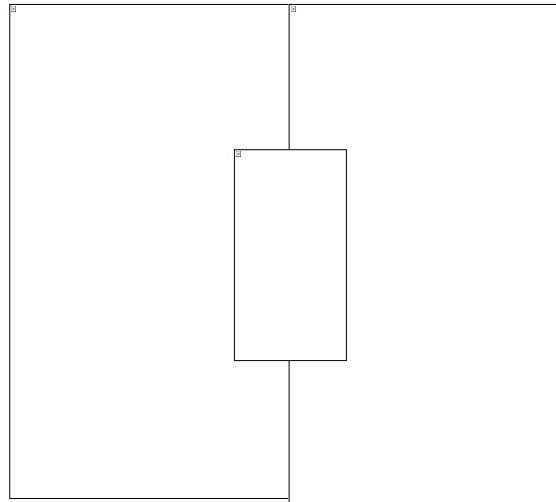


After these all-temp files were created during this are deleted leaving just the output.

Testing

I ran the pre-specified testing data through this section of my program. I do not think I needed to do any user validation as this feed would be directly provided by the camera so there was no need to test anything to this effect. My testing data produced three files (front_out,side_out, Ultraedge). This was as expected I have arranged the outputs as if they would be in the final program and placed them below.

Outputs:



Please note that Front & Side weren't written on the outputs. I placed these on manually for identification purposes in my report.

Evaluation

This sprint does everything I planned for it do at the start when I set out to make it but overlay everything together. This was because I discovered that ffmpeg wasn't very well suited to do this. My plan is to now overlay everything together in the main program.

As this isn't a front-end feature, I did not have to consider usability. However, if I were to make any changes, I would probably overlay the feeds in ffmpeg anyway as this would mean that the software only has to show 1 feed of video instead of 3. This would also reduce the level of performance required to run this program.

Sprint 2 (Quadratic Solver)

According to my plan this sprint must be able to:

1	Take in 3 sets of X, Y, Z Co-ordinates
2	Calculate and return the respective values of A, B, C of the quadratic
3	Be able to detect if ball is going to bounce (if $y \leq 0$)

For my project to work it needs to be able to figure out the final height of the ball to determine if this is less or more than the height of the stumps. As all projectiles follow a parabolic trajectory, I have decided to model the vertical height of the ball using a parabola (quadratic). The goal of this sprint is to calculate the equation of this parabola from a provided set of co-ordinates. Since only the vertical motion of the ball is parabolic, I only need to work with the Y and Z co-ordinates.

The code I create will be used multiple times and each answer for a, b & c will be stored. The final value used will be an average of all of these. As the general equation for a quadratic ($ax^2 + bx + c = 0$) has 3 unknowns each iteration must take in 3 co-ordinates and simultaneously solve them. For testing purposes, I have added validation. However, later in the project when I assemble the

backend from all the parts I will remove this and take values from a file. Below is the outline of how the maths will work.

Maths (Plan)

$$\begin{array}{r} az_1^2 + bz_1 + c = y_1 \\ -az_2^2 + bz_2 + c = y_2 \\ \hline (z_1^2 - z_2^2)a + (z_1 - z_2)b = (y_1 - y_2) \end{array}$$

Eliminate C from equations

$$\begin{array}{r} az_3^2 + bz_3 + c = y_3 \\ -az_2^2 + bz_2 + c = y_2 \\ \hline (z_3^2 - z_2^2)a + (z_3 - z_2)b = (y_3 - y_2) \end{array}$$

$$\begin{array}{r} \left(\frac{(z_1^2 - z_2^2)}{(z_3^2 - z_2^2)} \right) ((z_3^2 - z_2^2)a + (z_3 - z_2)b = (y_3 - y_2)) \\ \hline (z_1^2 - z_2^2)a + (z_1 - z_2)b = (y_1 - y_2) \\ \hline (z_3 - z_1)b = (y_3 - y_1) \end{array}$$

Multiply out and eliminate out A

$$(z_3^2 - z_2^2)a + (z_3 - z_2) \frac{(y_3 - y_1)}{(z_3 - z_1)} = (y_3 - y_2)$$

Sub back into previous equations

$$\left(\frac{(y_3 - y_2) - (z_3 - z_2) \frac{(y_3 - y_1)}{(z_3 - z_1)}}{(z_3^2 - z_2^2)} \right) = a$$

$$\frac{(y_3 - y_1)}{(z_3 - z_1)} = b$$

Rearrange to find A,B & C

$$- \left(\frac{(y_3 - y_2) - (z_3 - z_2) \frac{(y_3 - y_1)}{(z_3 - z_1)}}{(z_3^2 - z_2^2)} \right) z_1^2 - \frac{(y_3 - y_1)}{(z_3 - z_1)} z_1 + y_1 = c$$

Programming Techniques

In order to improve code legibility and simplicity I will use Object Orientated Programming (OOP) to do this sprint. Each equation will be an object that is made from an equation class. The equation class will have an attribute for each one of its' coefficients/variables. It will also have a method to multiply itself by a given factor.

Algorithms

The program will follow the flow of the maths above. There will be functions for: Taking Inputs, subtracting two equations & multiplying out an equation by a numerical factor. I have chosen these divisions because each bit has its own independent purpose. The results will then be printed.

Code

Variables and Data Structures

Name	Type	Purpose
x	Variable (in Equation Class) -Float	Supplied value of X co-ordinate observed.
y	Variable (in Equation Class) -Float	Supplied value of Y co-ordinate observed.
a	Variable (in Equation Class) -Float	Coefficient of a in equation
b	Variable (in Equation Class) -Float	Coefficient of b in equation
c	Variable (in Equation Class) -Float	Coefficient of c in equation
Equation	Class	The class to create equation objects that later will be solved simultaneously
equations_invalid	Variable - Boolean	Used for input validation (Turns false when input is valid to break while loop)
X	Variable – String	User inputs for X co-ordinates
Y	Variable - String	User inputs for Y co-ordinates

factor	Variable – Float	(In multiply method) A factor to multiply out an equation
original	Object	Equation to subtract from (In subtract function)
subtract	Object	Equation to subtract (In subtract function)
eq1	Object	First equation to simultaneously solve
eq1	Object	Second equation to simultaneously solve
eq3	Object	Third equation to simultaneously solve
r1	Object	Result of calculation 1
r2	Object	Result of calculation 2
r3	Object	Result of calculation 3
A	Variable – Float	Value of A
B	Variable – Float	Value of B
C	Variable – Float	Value of C

Testing:

What am I testing?	Input	Output	Passed?
Validation – Incorrect Input (Expecting float) Unit Test	“a”	Incorrect Input	✓
Multiply Method Unit Test	eqtest = Equation (1, 1) print(eqtest.multiply(2))	A: 4 B: 2 C: 2	✓
Subtract Method Unit Test	eqtest = Equation(1.0,1.0) eqtest2 = Equation(2.0,2.0) print(subtract(eqtest2, eqtest))	A:3 B:1 C:0	✓
Whole Function (Expected Answer = 0, -1,4) Integration Test	X = 1,2,3 Y = 3,2,1	-0.0, -1.0,4.0	✓

However, after completing this I realised there was a better, neater, and more efficient way of solving for A, B and C using matrices. This also makes the code much more legible. Here is what the maths looks like:

$$\begin{pmatrix} y_1 = az_1^2 + bz_1 + c \\ y_2 = az_2^2 + bz_2 + c \\ y_3 = az_3^2 + bz_3 + c \end{pmatrix} \Rightarrow \begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix} = \begin{bmatrix} z_1^2 & z_1 & 1 \\ z_2^2 & z_2 & 1 \\ z_3^2 & z_3 & 1 \end{bmatrix} \begin{bmatrix} a \\ b \\ c \end{bmatrix}$$

$$\begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix} \begin{bmatrix} x_1^2 & x_1 & 1 \\ x_2^2 & x_2 & 1 \\ x_3^2 & x_3 & 1 \end{bmatrix}^{-1} = \begin{bmatrix} a \\ b \\ c \end{bmatrix}$$

Programming Techniques

The matrices would be represented by nested arrays as the dimensions are known and fixed however Python only has lists so I will be using lists. I will be using a lot of decomposition and abstraction to help develop the inverting of the matrix. I will also need to use an exception catching statement (try... catch statement) as some matrices are singular (i.e. they can't be inverted). This means that there is no solution. Ideally this should never occur however in case if the true location of the ball is slightly off due to a random error (e.g. due to motion blur) there's a small chance that there will be no solution to that set of co-ordinates. I will also use for statements.

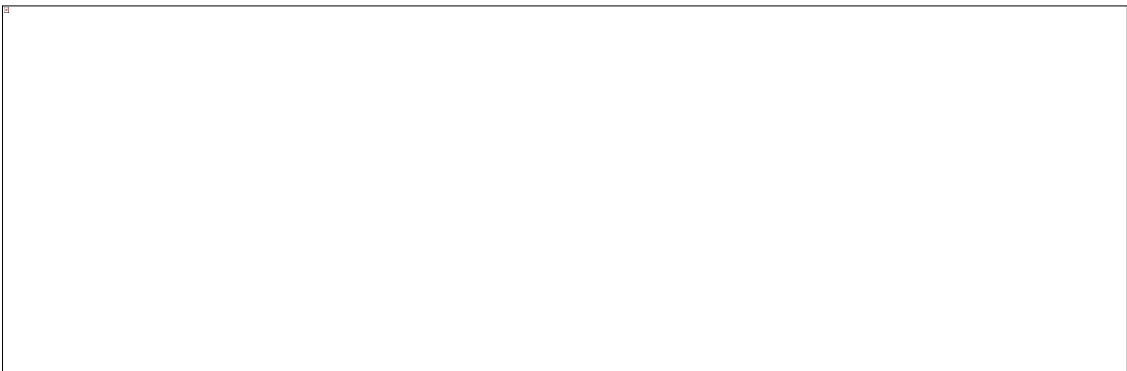
Plan

To inverse a 3x3 matrix you must first Transpose the matrix (swap the rows and columns around). Then you must find the Matrix of Minors (a matrix of the minors of every value). Then you must simply work out the cofactors (by multiplying the Matrix of Cofactors). Finally, you must multiply by the coefficient (1/the determinant). The input is/will be unvalidated because all values will be entered programmatically using a computer. There will be function for each of the steps. The whole process is outlined by the flow chart below. Each function is colour coded.

The whole thing will be enclosed by a try statement to catch any errors when the data produces a singular matrix due to small inaccuracies in data collection or a false positive on the ball location.



InverseMatrix Code:



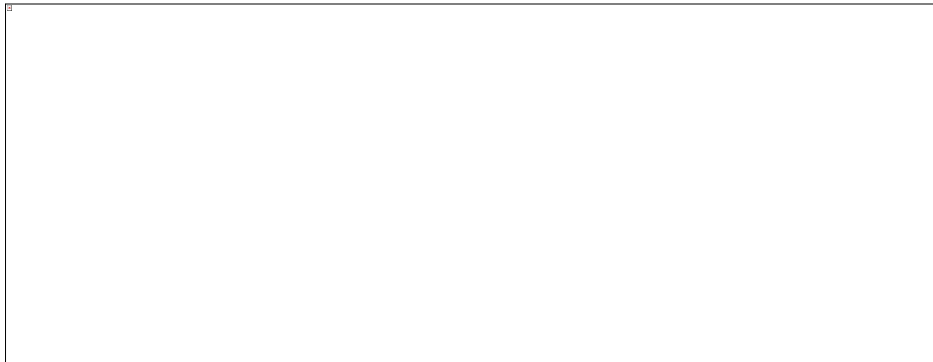
Code calls all functions and prints final values in an array. Enclosed in a try except statement. Will alert if matrix is singular (i.e. no solution)

GetMatrix()



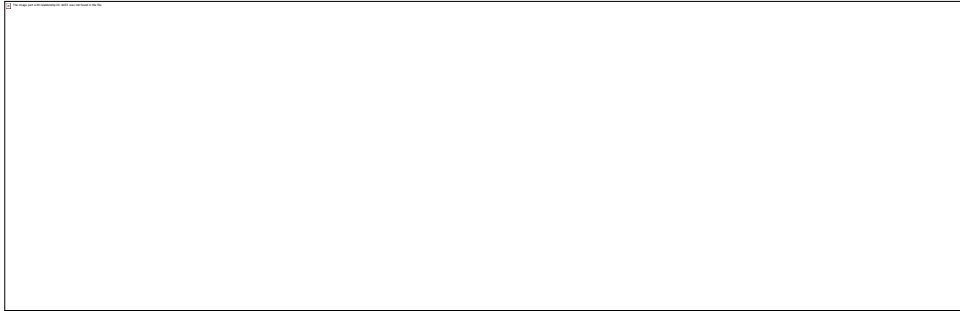
This code is temporary and mainly for testing purposes. This is because in the final integration steps where I join all the code together I will make this code import its values from a file. Its parameters are (x, y). X is the amount of rows whereas y is the amount of columns. I have abstracted a Matrix to be a list that has a child list for each row. I couldn't use Rows and Columns as variable names as these are needed in the function. It uses nested for loops to first loop through every row and then every element in said row. I have added a while loop for validation.

Transpose(Matrix)



This function will transpose any n x n matrix. It cycles through every cell in the matrix and returns a matrix in which the column and row indexes are switched. I have had to use the `deepcopy()` function to create a copy of the input parameter as python passes by value instead of by reference. Unfortunately, I learned this the hard way in debugging when the individual functions found the right answer individually but when the wrong answer when they were all combined.

MatrixOfMinors(Matrix)



The MatrixOfMinors of function goes through every cell in the Matrix and calculates it's minor (a minor is the determinant what's left of the matrix where the row and column of the target element is removed from the matrix) . Since I have abstracted the Minor calculation to another function all this function needs to do is iterate through the matrix and call this function.

Minor(Row,Column,Matrix)



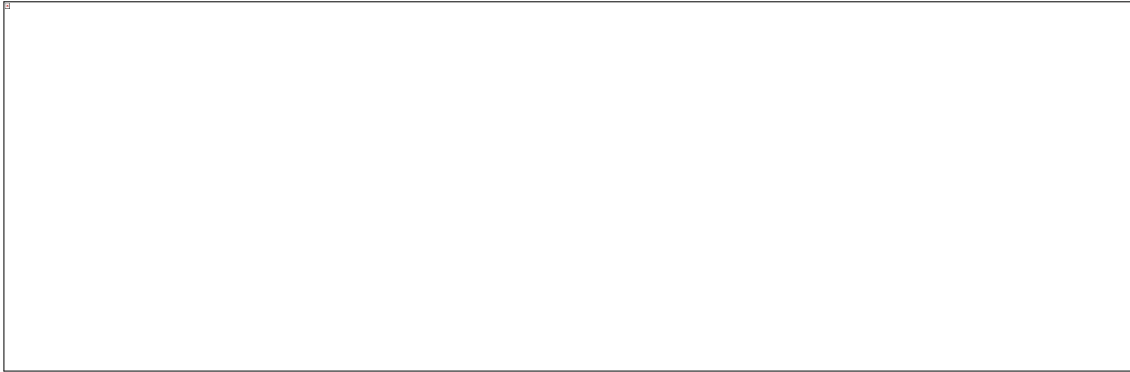
The Minor() function will find the Minor of any given element in a matrix. To do this it must remove the entire column and row where in which the targeted element existed in. Then it must calculate the determinant of this. I have taken Row and Column in as parameters to do this. Because of my abstraction to remove rows all I have to do is pop them from the list. However, to remove columns I must iterate through every row and pop out the item in that column. Due to my previous abstraction all I then have to do is call the Determinant function.

Determinant(Matrix)



The determinant is a singular scalar value that can express certain properties about a matrix. This is needed to inverse a matrix. This function only works for 2x2 matrices as this as far as it needs to go for my project. The determinant of a 2x2 matrix is simply $ad - bc$ where the matrix looks like: $\begin{bmatrix} a & b \\ c & d \end{bmatrix}$

Cofactors(Matrix)

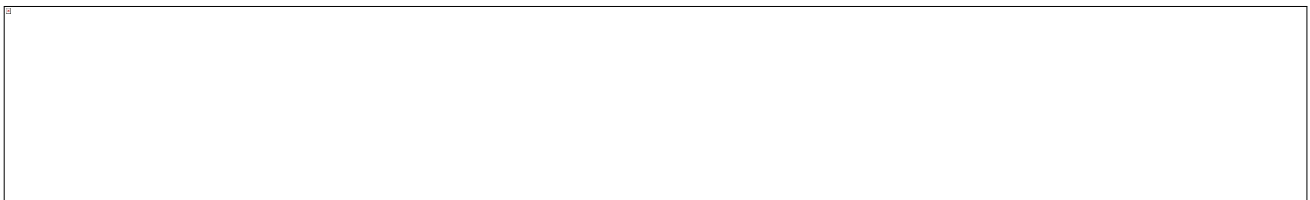


Cofactors works out the Matrix of Cofactors for any $n \times n$ matrix. The matrix of cofactors is the original matrix multiplied out by the following matrix:

$$\begin{bmatrix} + & - & + \\ - & + & - \\ + & - & + \end{bmatrix}$$

It does this by multiplying each element by -1 to the power of its' row index plus it's column index. It uses a copy of the matrix as the original values are still needed for other functions.

Coefficient(Matrix)



Returns the coefficient of matrices inverse. It does this by calculating the determinant (for a 3x3 matrix). Then taking the reciprocal.

The formula for the determinant of a 3x3 matrix is simply:

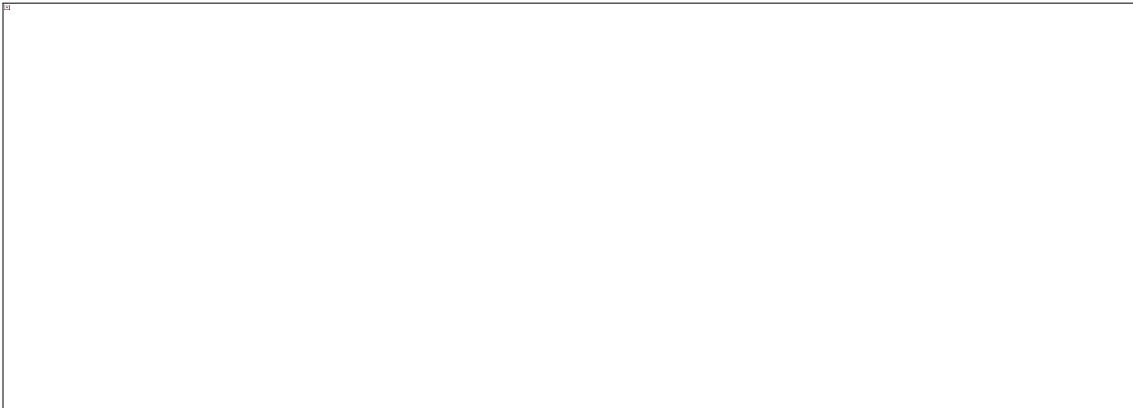
$$(a * \text{Minor}(a)) - (b * \text{Minor}(b)) + (c * \text{Minor}(c))$$

where

$$\text{Matrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix}$$

Thanks to my previous abstraction this algorithm simply calls then multiplies, adds, and subtracts values returned by the Minor function.

ScalarMultiply(Matrix,Coefficient)



The purpose of this angle is to multiply out a Matrix by a single-dimensional coefficient factor. I am using it to multiply the Matrix Of Cofactors by the Coefficient to work out the final Inverse. It does this by iterating through every element in the matrix (using nested while loops) and multiplying them out by the coefficient. It then returns the new matrix.

Variables & Data Structures

Name	Type	Purpose
Input	Array(Array[string], Length = 3)	Store the users inputted values so it can convert into a float based matrix for calculations
x,y	Integer	The original parameters of the GetMatrix() function that determine the dimensions of the matrix.
Rows	Integer	Number of rows in the matrix
Row	Array(Float, Length = 3)	Value of the current row (In a for loop running through each row)
Columns	Integer	Number of columns in the matrix
Valid	Boolean	Temporary user validation
UserInput	String	Takes the input of the user for each element in the matrix
ad	Float	Element [0,0] * Element [1,1]
bc	Float	Element [0,1] * Element [1,0]
row	Array(Float, Length = 3)	A row in the matrix
Column	Array(Float, Length = 3)	A column in the matrix
det	Float	Determinant of the whole matrix not just a minor of the matrix.

Transposed	Array(Array[Float], Length = 3)	Copy of original matrix for calculations
n	Integer	Dimension of n*n matrix
r	Integer	Index of row in matrix
c	Integer	Index of column in matrix
MinorMatrix	Array(Float, Length = 3)	Copy of original matrix for calculations
Cofactor	Array(Float, Length = 3)	Copy of original matrix for calculations
dotproduct	List	Is the result of the dot product between two matrices.
temp	Float	A temporary variable to store the value of the current dot product between two pairs
YMatrix	List	Is the matrix of Y values of the ball
Input	Array(Float, Length = 3)	Returned value of GetMatrix() – Function that gathers and validates user input
transposed	Array(Float, Length = 3)	Returned value of Transpose() – Function that flips row and column of the matrix
minors	Array(Float, Length = 3)	Returned value of Transpose() – Function that calculates the matrix of minors
cofactors	Array(Float, Length = 3)	Returned value of Transpose() – Function that calculated the matrix of cofactors.

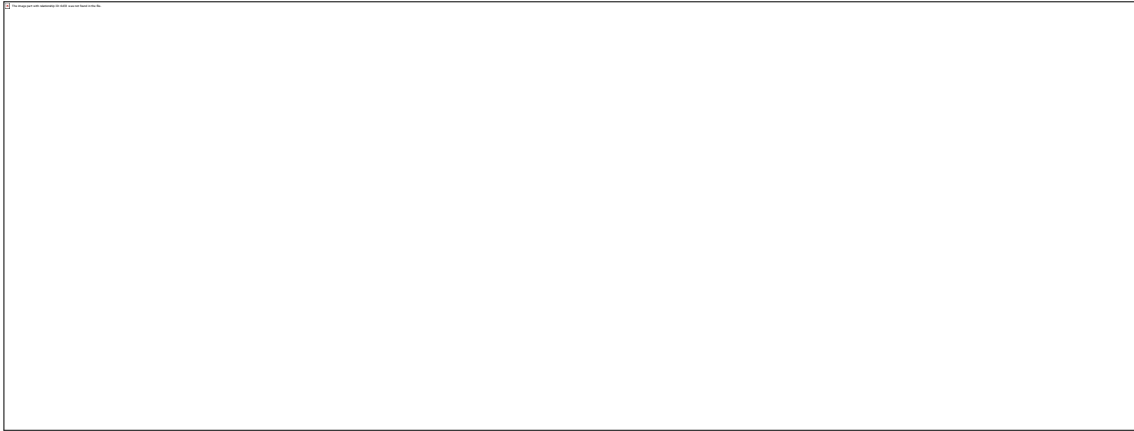
Testing the final algorithm:

What I am testing?	Input	Output	Passed?
GetMatrix() Unit Test	1,2,3,4,5,6,8,9	[[1,2,3],[4,5,6],[7,8,9]] – <i>Literal</i> $\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$ – <i>Abstracted</i> <i>*from now all matrices will be written in abstracted form</i>	✓
Transpose() Unit Test	$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$	$\begin{bmatrix} 1 & 4 & 7 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$	✗
MatrixOfMinors() Unit Test	$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$	$\begin{bmatrix} -3 & -6 & -3 \\ -6 & -12 & -6 \\ -3 & -6 & -3 \end{bmatrix}$	✓
Minor() Unit Test	$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$ Row: 0 Column: 0	-3	✓

Determinant() Unit Test	$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$	-2	✓
Cofactor() Unit Test	$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$	$\begin{bmatrix} 1 & -2 & 3 \\ -4 & 5 & -6 \\ 7 & -8 & 9 \end{bmatrix}$	✓
Coefficient() Unit Test	$\begin{bmatrix} 1 & 0 & 5 \\ 2 & 1 & 6 \\ 3 & 4 & 0 \end{bmatrix}$	1	✓
ScalarMultiply() Unit Test	$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}, 2$	$\begin{bmatrix} 2 & 4 & 6 \\ 8 & 10 & 12 \\ 14 & 16 & 18 \end{bmatrix}$	✓
Validation – Incorrect Input (Expecting float) Unit Test	$\begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix}$	Input: Row>0 Column>0: a Input: Row>0 Column>0: b Input: Row>0 Column>0: c (Doesn't move on to next element till valid input is entered)	✓
Singular Matrix Integration Test	$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$	No Solution	✓
Matrix of known inverse (Whole function test) Integration Test	$\begin{bmatrix} 1 & 0 & 5 \\ 2 & 1 & 6 \\ 3 & 4 & 0 \end{bmatrix}$	$\begin{bmatrix} -24 & 18 & 5 \\ 20 & -15 & -4 \\ -5 & 4 & 1 \end{bmatrix}$ Co-efficient = 1 Only after Fix of transpose	✓

Fixing the Transpose function

After further testing I realised that it was only Transposing the first row/column so I just added another for loop to enclose the existing code so it would run once for each row and this ended up fixing the code.

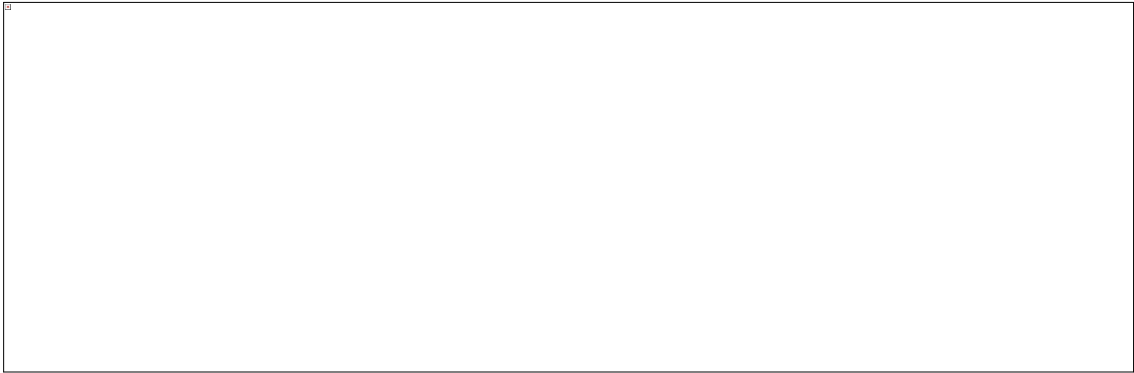


The code then worked. Here is the update to the testing table:

What I am testing?	Input	Output	Passed?
Transpose() Unit Test	$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$	$\begin{bmatrix} 1 & 4 & 7 \\ 2 & 5 & 8 \\ 3 & 6 & 9 \end{bmatrix}$	

Updating the cofactors code

After re reading my code I determined that this was overly complicated and hard to read so I redesigned this to be easier top read and use less computational time.



The code is still very similar. I only changed lines 6-8. I did this by multiplying each element by $-1^{(R + C)}$ (Its row index + its' column index). This looks much easier then the $(3R + C1)$ pattern that was both confusing and unnecessary when a more elegant solution like this exists.

Dot product multiplication

To calculate the final answer, from the inversed matrix we must multiply the inversed matrix by the original values for Y (as described in the original maths plan above). However, matrices don't follow the normal rule for multiplication. They follow a form of multiplication called dot product multiplication. Dot product multiplication allows you to multiply two multi-dimensional objects by each other.

$$\mathbf{a} \cdot \mathbf{b} = \sum_{i=1}^n a_i b_i$$

So final dimensions are Width A x Length B

Algorithmic plan:

Programming Techniques Used

Functions, parameters, iteration, and variables have been used as programming techniques in this algorithm. Furthermore, I used 'thinking ahead' to plan what inputs are needed to this function (Matrix1 and Matrix2) along with decomposition to decide to make this into its own function.

Code

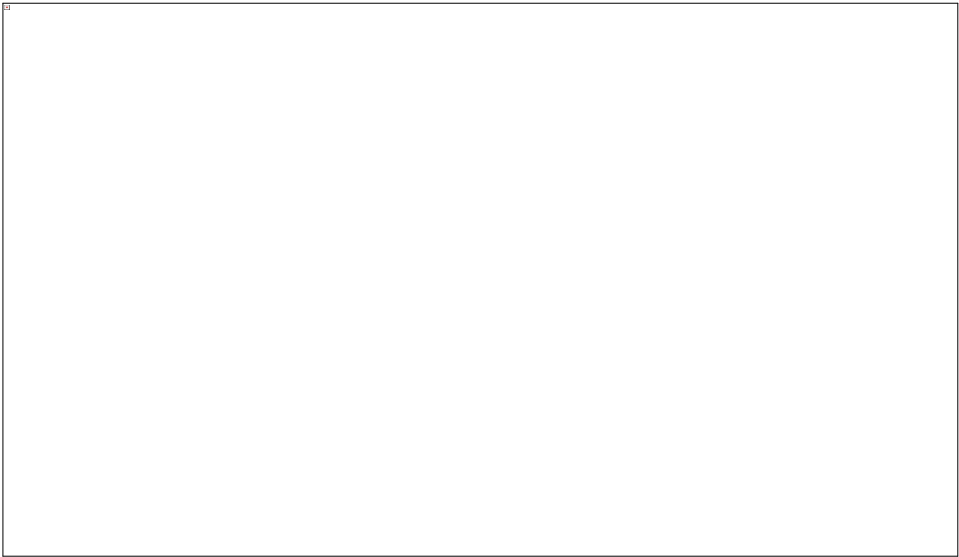
The function of this code is to multiply two matrices (using the dot product method). It does this by iterating through every element in both matrices and then multiplying them by each other. It then stores and returns these results in a list.

Testing

What I am testing?	Input	Output	Passed?
DotProduct() Unit Test	$\begin{bmatrix} 1 & 0 & 5 \\ 2 & 1 & 6 \\ 3 & 4 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$	[1, 2, 3, 0, 0, 0, 5, 10, 15, 2, 4, 6, 1, 2, 3, 6, 12, 18, 3, 6, 9, 4, 8, 12, 0, 0, 0]	✗

Fixing the DotProduct() function

The programs faults lie within my misunderstanding of the dot product formula that was shown above at the start. The dot product is between rows and column rather then elements. This algorithm now fixes that.



Testing

What I am testing?	Input	Output	Passed?
--------------------	-------	--------	---------

DotProduct() Unit Test	$\begin{bmatrix} 1 & 0 & 5 \\ 2 & 1 & 6 \\ 3 & 4 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$	$\begin{bmatrix} 16 \\ 22 \\ 11 \end{bmatrix}$	
----------------------------------	---	--	---

Evaluation

In the end, all the unit and integration test have passed. However, I didn't achieve the 3rd criteria (determine if the ball is going to bounce). This is because I felt like this code would be better place in the "Determining the height of the ball" sprint (Sprint #3). The other two criteria (Taking in 3 sets of X,Y Co-ordinates and calculating the values for A,B,C) were both achieved flawlessly.

The end product that I achieved was much better than the original plan as it is more readable and takes less processing power. However, if I were to remake this sprint again, I would have made the determinant function work for matrices of all sizes instead of just 3x3 matrices. This would allow me to make it into its own library for later projects.

As this is a backend sprint the usability did not need to be considered but was improved as the calculation time decreased then what was previously thought.

Sprint 3 Determine Height of ball

According to my plan this sprint needs to:

1	Estimate and Calculate/Retrieve stored Co-efficient of restitution
2	Use Coefficient of Restitution to calculate maximum height
3	Use this maximum height in conjunction with the bouncing point to work out final parabolic trajectory
4	Work out final vertical position

Design

This sprint must first determine if the ball will bounce (as I decided not to do this in the previous sprint because It makes more sense to do it here). We will do this by checking if the root of the quadratic (when the quadratic = 0) falls before the end of the pitch (in m). This is because the quadratic will = 0 when the ball hits the ground. If it does then we must calculate the new equation. Then we will recursively check if it bounces again before the stumps. If this recursion happens more than once, then we will break the recursive cycle as the ball isn't allowed to bounce more than twice.

Once we have determined the final equation for the height of the ball and if it's legal. We can determine or calculate the balls height depending on the circumstances. If the ball bounces once and then hits the batters pads then we can calculate the height of the ball using the previous sprint but with new values for X,Y. However, if it bounces more than once then we will also have to use the coefficient of restitution to determine the new equation. The coefficient of restitution is the ratio of resultant initial and final velocities. It used to determine the final velocity after a bounce/collision.

To predict the coefficient of restitution I could predict a value interpolating previously measured values of the coefficient on that pitch with that ball. We would do this by detecting all bounces and not just when there is a review. We could then record a value for the initial and final velocity. Then we can calculate based the estimate based off that.

However, upon further research I realised that the coefficient of restitution is affected by lots of different factors such as temperature, humidity, wind direction and/or speed, pressure etc. To effectively count for all of these we could record all the factors along with initial and final velocity and then use a Machine Learning algorithm such as the K-Nearest-Neighbour (KNN) algorithm.

The KNN algorithm for regression works by making and populating a multi-dimensional graph with all the nodes it has received. When give a node it doesn't know it finds the values nearest to this unknown across all dimensions (or it's neighbour). This works well because there is no training stage so the algorithm can be ready to go very quickly. Furthermore, as it is supervised It will learn as it goes along. However, the algorithm suffers from the curse of dimensionality. The curse of dimensionality is when the accuracy of an algorithm goes down as the number of dimensions goes up. This means we won't be able to include all the affecting factors if we want an accurate prediction. Additionally, as the number of nodes grows the algorithm becomes slower so it will start to take a while for it to load the answer especially as processing is done locally on a low powered mobile device.

This led me to the conclusion that the best way to estimate the coefficient of restitution is to use the calculate the coefficient from the first bounce if the ball has already bounced or the previous ball if it hasn't already bounced (i.e., it's a low full toss). This is because the factors will mostly all be the same. Although, problems occur if the full toss is the first ball of the day if the previous ball was an outlier or even if the previous ball gets lost because someone hit it over the fence. However, I believed this to be the best compromise between both methods.

The coefficient of restitution (e) forms a limited range reciprocal graph when plotted against initial velocity. This is because $e = \frac{v_{final}}{v_{initial}}$. If we were to graph e against initial velocity (since we always have initial velocity) we would have $y = \frac{v_{final}}{x}, \{x > 0\} \{0 < y < 1\}$. This produces a graph that



looks like this:

Please note that I used $v = 1$ in this graph

X (Initial velocity) will always be above 0 because we are only calculating if the ball bounces upwards off the floor not if it bounces downwards of a bat or pad. Whereas the coefficient of restitution will always be between 0 (perfectly plastic) and 1 (perfectly elastic). This means that if we have an estimate for the coefficient of restitution and the initial velocity then we could work out the final velocity.

Pseudo code

Finding Velocity

As previously discussed in order to work out the final velocity and thus height we first must know the initial velocity and the coefficient of restitution. This code finds the initial velocity (across one plane) by using the simple $v = \frac{s}{t}$ formula. As we know that the camera records at 30fps we can simply divide the number of frames by 30 to get the initial velocity in $cm.s^{-1}$.

Finding the 2nd Root

In order to find the equation of the graph we need to the value of both of the roots. I have used a very quick and simple method to calculate this, and it unfortunately doesn't take into account a loss in horizontal velocity when the ball hits the ground due to friction.

The code makes use of the following maths:

$$E_k = E_{GPE},$$
$$mgh = \frac{1}{2}mv^2$$

$$h = \frac{(e * v_{initial\ vertical})^2}{2g}$$

$$v^2 = u^2 + 2as$$

$$0 = (e * v_{initial\ vertical})^2 + 2a * \left(\frac{(e * v_{initial\ vertical})^2}{2g} \right)$$

$$0 = (e * v_{initial\ vertical})^2 + a \left(\frac{(e * v_{initial\ vertical})^2}{g} \right)$$

$$-(e * v_{initial\ vertical})^2 * \left(\frac{(e * v_{initial\ vertical})^2}{g} \right)^{-1} = a$$

$$a = \frac{v - u}{t}$$

$$-\frac{(e * v_{initial\ vertical})^2 * g}{(e * v_{initial\ vertical})^2} = \frac{0 - (e * v_{initial\ vertical})}{t}$$

$$t = -(e * v_{initial\ vertical}) * \left(\frac{(e * v_{initial\ vertical})^2 * g}{(e * v_{initial\ vertical})^2} \right)^{-1}$$

$$t = -\frac{(e * v_{initial\ vertical}) * (e * v_{initial\ vertical})^2}{(e * v_{initial\ vertical})^2 * g}$$

$$t = -\frac{(e * v_{initial\ vertical})}{g}$$

$$Vertex_x = -\frac{(e * v_{initial\ vertical})}{g} * v_{horizonta}$$

$$Root2_x = (Vertex_x - Root1_x) + Vertex_x$$

The maths first equates Gravitational Potential energy (GPE) and Kinetic Energy as these are equal when the maximum height of the ball is reached. The mass cancels out straight away and then I rearrange for the maximum height. I then use SUVAT to work out the acceleration the ball experiences between the instant of bouncing and reaching its new maximum height. From, this value of acceleration I can work out time taken for this to happen. If I multiply this time by the Horizontal velocity, I get the x value of the vertex. Then, I make use of the fact that parabolas are symmetric to work out the value for the 2nd Root.

Finding the coefficients of the new equation

The purpose of this function is to calculate the coefficients of the new equation. I achieved this by using the roots to expand out from factorised form and then use my vertex co-ordinates to work out my value of a.

The maths I used is below:

$$\begin{aligned}
 0 &= a[(x - \text{Root1}_x)(x - \text{Root2}_x)] \\
 0 &= a[x^2 - (x * \text{Root1}_x) - (x * \text{Root2}_x) + (\text{Root1}_x * \text{Root2}_x)] \\
 y &= a[x^2 - x(\text{Root1}_x + \text{Root2}_x) + (\text{Root1}_x * \text{Root2}_x)] \\
 \text{MaxHeight} &= a(\text{Vertex}_x^2 - \text{Vertex}_x(\text{Root1}_x + \text{Root2}_x) + (\text{Root1}_x * \text{Root2}_x)) \\
 a &= \frac{\text{MaxHeight}}{(\text{Vertex}_x^2 - \text{Vertex}_x(\text{Root1}_x + \text{Root2}_x) + (\text{Root1}_x * \text{Root2}_x))} \\
 b &= a * -(\text{Root1}_x + \text{Root2}_x) \\
 c &= a * -(\text{Root1}_x * \text{Root2}_x)
 \end{aligned}$$

Finding the final Height

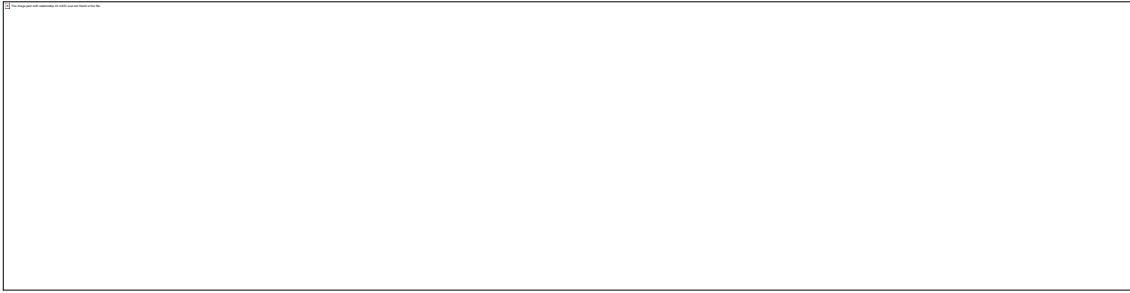
Subs in 500 to the newly generated polynomial. 500 is chosen to be subbed in as the front-facing camera is 5m away from the stumps and the relative co-ordinate system is in cm.

Code

Find1DVelocity()

This function is very similar to the pseudocode because it's very simple so doesn't require any further description.

SecondRoot()



Again this function is very similar to the pseudocode as most of the complexity for this sprint lies within the maths rather than the execution of the code.

Coefficients()



When I was designing my pseudo code, I made a major oversight when I missed out the minus sign from coefficient b . However, I fixed this in my final code before testing.

GetHeight()



The only difference here is that I decided to use one list instead of 3 separate parameters. I plan to update this sprint when I do Sprint 7 to launch everything else from it.

Note on determining bounces



I did determine the pseudo code above to determine if the ball were to bounce and if so to call the function to determine it's new equation. However, because this would need access to both planes of co-ordinates instead of just 1 I decided it would be better to migrate this function into Sprint 7 where that data would already be needed anyway. This is to save space in memory from being unnecessarily used.

Programming Techniques

This sprint used a lot of abstraction to break down the complicated maths into simple calculations by cancelling out variables and equating formulas. Knowing where to break into functions required a large amount of decomposition. Furthermore, knowing when to break something down that could be 1 line in to two or three for readability also made good use of my decomposition abilities. My extensive pre planning also required me to think ahead.

This sprint required me to work effectively with external libraries, declare variables and use iteration as well.

Variables & Data Structures

Name	Type	Purpose
coordinates	List	Store the previous co-ordinates of ball
displacement	Float	Amount of vertical displacement the ball has undergone
time	Float	Total time that ball was in video
initial_vertical_velocity	Float	Initial velocity in the vertical plane before bounce
horizontal_velocity	Float	Velocity in the horizontal plane
e	Float	The coefficient of restitution
root1_x	Float	X value of 1 st root
vertex_x	Float	X value of vertex
root2_x	Float	X value of 2 nd root
max_height	Float	The heighth point the ball would reach (y value of vertex)
y	Float	Partial equation
a	Float	Value of coefficient 'a'
b	Float	Value of coefficient 'b'
c	Float	Value of coefficient 'c'
coefficients	List	List containing all 3 coeffcients

Testing

What I am testing?	Input	Output	Passed?
Find1Dvelocity () Unit Test	[62.392137373746856, 62.505254431922516, 59.531891759876565, ... 56.04142253617045, 55.94446505773417, 55.9121458982554, 55.9121458982554]	12.025m/s	✓
Does the code work() Integration test	[10,7,5]	A = -5,B = 2.5, C = 4.87	✓

Passed because within expected range

Evaluation

Whilst my code does output a final height of the ball at the end of the pitch (which was my overall goal for this sprint) it does not do this how I planned or to the degree of accuracy I would like. Because of this not all my criteria were met.

For example, this doesn't predict a value for the coefficient of restitution (e) but rather uses a researched value I found on the internet. This is because of time and complexity restraints. As I discussed to accurately predict the value for e I would have had to record every ball and needlessly run it through code. This would drain the batteries of the cameras being used and make the product less usable to the user. Furthermore, it would require a system to live stream two separate high resolution video streams to a central processing server in sync and have a value returned before the next ball is bowled. This would be very tight timings and would probably further annoy players as well as require much better internet connectivity speeds outdoors.

Furthermore, this code also doesn't determine if the ball is going to bounce. Only the equation for if it does. Although there was a logical reason for this, and this functionality will still occur. This means that this criteria failure is not as bad.

Sprint 4 Determine Lateral Movement of ball

Design

According to my plan this sprint needs to:

1	Calculate Movement before bounce (Linear as through air)
2	Calculate Movement after bounce (Re-analyse footage after parabola hits root)
3	Return Final Position of ball

As the movement of the ball laterally is linear (assuming constant windspeed when the ball is in the air). This is because other than what spin/swing the bowler puts on the ball and the wind not the forces are impact the ball when it's in the air.

Pseudocode

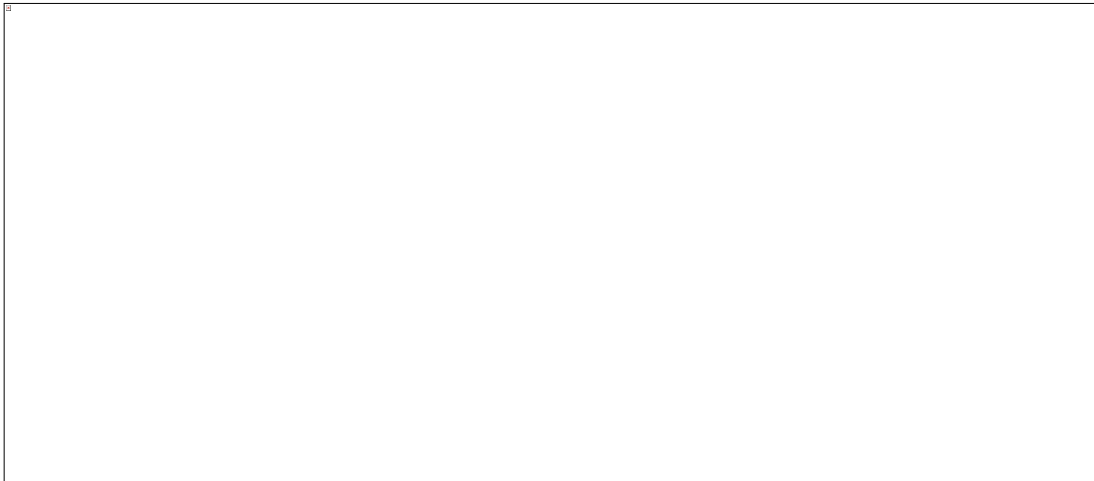


I plan to use the *sci-kit-learn* library to build a linear regression model. This is because it is the simplest and most easy way. *Sci-kit-learn* uses the least squares regression for approximation.

Programming Techniques

This code is very simple so did not require me to use many programming techniques. However, one technique I used was abstracting the complexity on to an external library instead of programming it all myself.

Code



The code is responsible for predicting the lateral movement of the ball. I have modelled this to be linear as there is no other forces impacting on the ball other than air resistance and the magnus force. I have chosen to ignore both as it would make the project a lot more complex for not a lot more accuracy.

Variables and Data Structures

Name	Type	Purpose
x_coordinates	List	Parameter for list of relevant x co-ordinates
y_coordinates	List	Parameter for list of relevant y co-ordinates
x	Numpy array	2 dimensional (although 1 is blank) NumPy array of x co-ordinates for graphing purposes
y	Numpy array	NumPy array of y co-ordinates for graphing purposes
model	Sci-kit-learn regression model	Linear regression model from the sci-kit-learn library

Testing

What I am testing?	Input	Output	Passed?
If the sci-kit learn model works? Unit Test	X = [1,2,3,4] Y = [3,5,7,9] ($y = 2x + 1$)	Coefficient:2 Intercept: 1	✓
Does it return the right answer? Integration Test	<i>Real test data values extracted from a video</i>	223.7887877	✗

This failed because the prediction for the ball was way out of bounds.

Fixing the code

At first the failure of this code stumped me as I couldn't see why it was failing when the unit test had passed but then I remembered I used the front camera out values, so I had to use the width of the pitch not the length. So, I swapped out the 500 for a 200.

Updated code:

```

# Updated code

```

Retesting

What I am testing?	Input	Output	Passed?
Does it return the right answer? Integration Test	<i>Real test data values extracted from a video</i>	48.0467593	✓

Evaluation

My code does fulfil the final point in my criteria of determining the final position of the ball but does not split the segments pre and post bounce. This is because of the assumption I made in the last sprint that horizontal velocity remains constant even after the bounce. In reality this is a simplification and will lead to small inaccuracies but I did not see the point in chopping and changing my assumptions of the model between sprints as this would only further complicate things.

Sprint 5 (OpenCV Object detection)

As this is much larger sprint then the previous two, I believed this needed further planning. Furthermore, as I had not used OpenCV before this was an ideal opportunity to research how other developers had created Object Recognition code and to learn best practices.

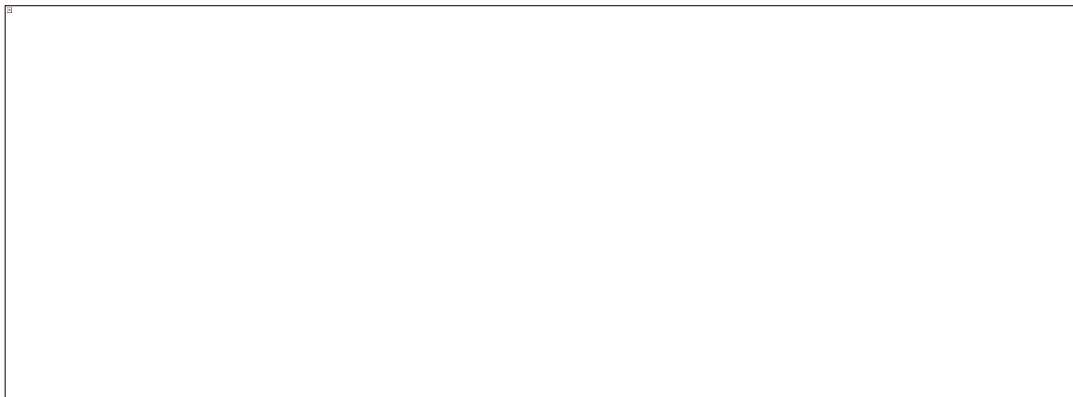
Design

According to my plan this sprint needs to:

1	Take in footage and split into frames
2	Apply any filters or transformations necessary for Computer Vision (e.g. RGB -> HSV)
3	Identify Ball through colour thresholding
4	Identify Stumps through colour thresholding
5	Contour two objects and export

I was planning to use OpenCV to detect the ball and the stumps. I chose this library as it is the most widely used and available algorithm so there would be lots of support available online if I were to run into any problems. After some research, I realised the best way to detect objects was to split a video input into frames and then use colour thresholding to determine where the objects were in the image. An alternative method would be to use machine learning however I believe that this would be excessive and not process any additional benefits to colour thresholding.

Template Algorithm (Pseudocode) for detecting the object:

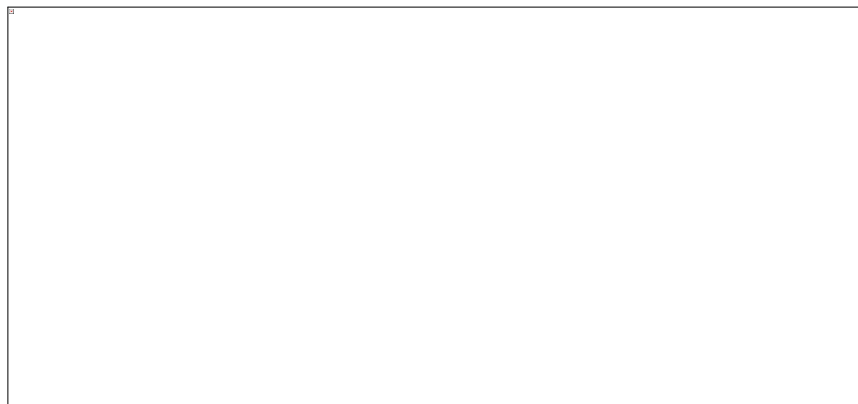


Testing Data



My code

Initialisation



The first few lines of my code are used to import the OpenCV and numpy libraries (numpy is a dependency of OpenCV). They also set the values of global variables (such as the upper and lower bounds of the ball and stumps for thresholding later. It then imports the test video feed from a file.

It also creates two new text files to export out the detected contours for use in other sections of my code.

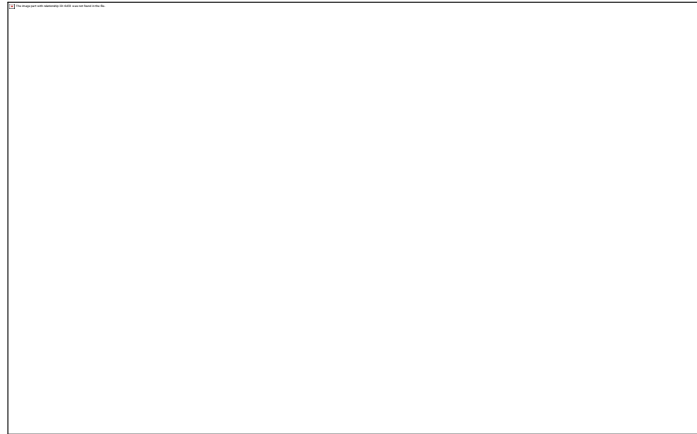
The rest of the code as a whole



As you can see the rest of the code is enclosed in a while loop. This is just to iterate through each frame in the provided video loop. The try statement is there to catch the error when the final frame of the video is opened (as this has 0 width and 0 height it causes an issue with my resizing code).

To best explain the rest of this code, I will break it into smaller sections to explain what each bit does. I did not break this into discrete functions in the actual program as I felt it would be counter-productive when trying to make the code more readable/understandable.

Exit Tracking()



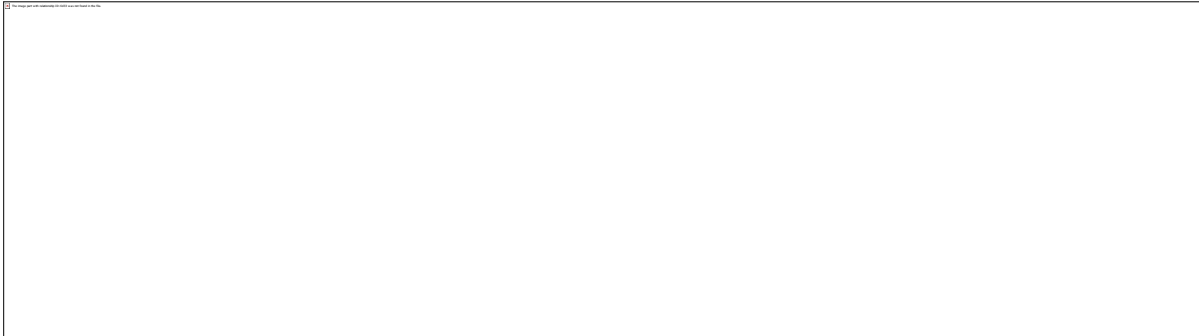
This function is called when the code needs to stop/close. It closes all open windows, removes the video from RAM as well as closes and saves my export files. The code will either need to stop/close when the software has completely run through the feed or if the emergency stop key is pressed (q). This was mainly included for testing purposes, but I left it in for troubleshooting and to cut feeds off manually in case of software malfunctions.

Resize and colourise the video



The `ret, frame = video.read()` reads the next frame of the video. I then converted this frame into a BGR image this is because OpenCV uses BGR as it's colour scheme. I then resized the frame to 30% the height and width of the original due to performance limitations of using the full resolution video.

Find Contours



The code first finds all the pixels that fall between the pre specified colour bounds for the ball and stumps. It then denoises these to remove any flickers of small groups of pixels due to changing light. It then uses the `cv2.findContours()` function to find contours (curves/shapes that fit the colour bounds). The ball uses simple chain approximation (as there is only one point on the ball being tracked). Whereas the stumps use the 'Teh – Chin' approximation method. This only returns the extreme points(boundaries) of the main contours. I have used this because we only need to know the height of the stumps so we can store a lot less data.

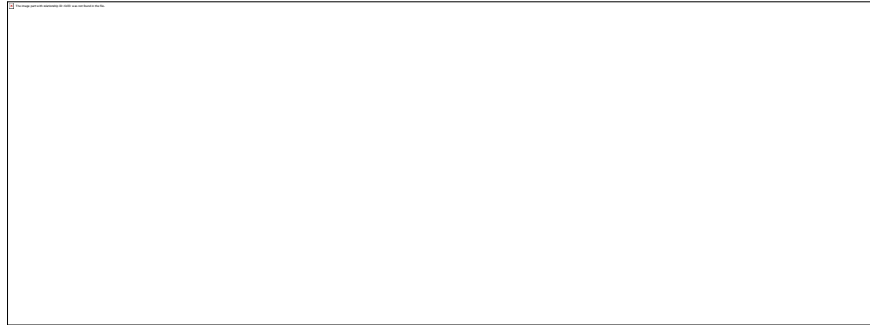
Storing & Drawing the contours



The code first exports the detected coordinates for the contours of the ball and stumps to text files along with '/' as a delimitator character. This is so they can be used by other parts of my program like the Relative coordinates system.

This part of the code is also responsible for drawing the contours on top of the video feed. This is mainly for debugging and troubleshooting purposes and thus will be removed in the final build of the project. The ball gets drawn on in green and the stumps get drawn on in blue(due to the BGR colour space).

Show the videos



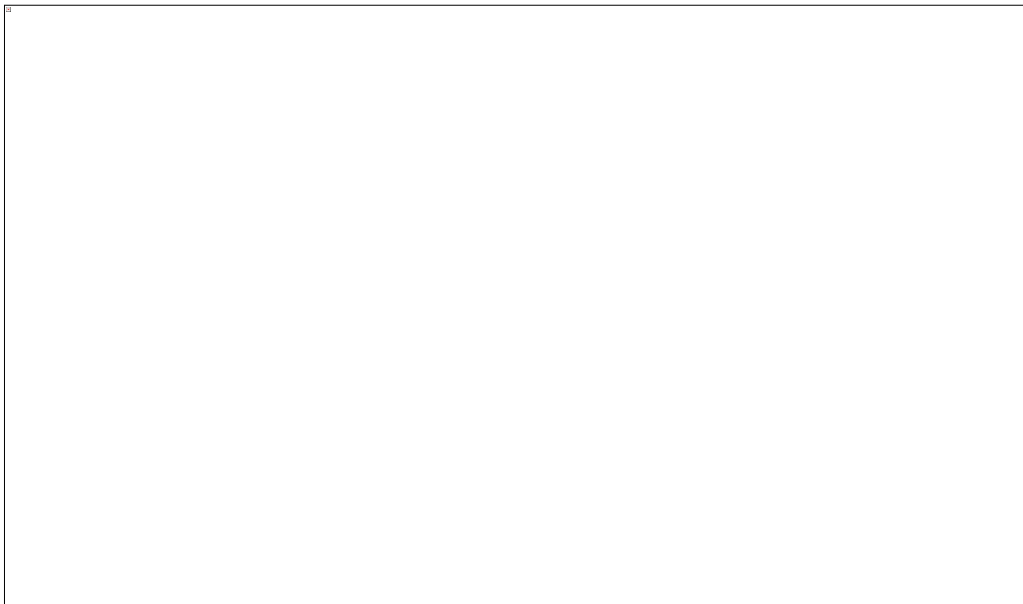
This code shows some of the videos produced by the algorithm in the process of trying to determine the location of the ball and stumps. It shows the masks (binary images of what falls in the prespecified colour thresholds) as well as the resized video that has been overlayed with the contours on it. However, this code is mainly for troubleshooting so it will likely be removed when I assemble everything together.

Programming techniques Used

I abstracted my development by using ready made libraries such as OpenCV and NumPY to make my development much easier. I also had to use decomposition to break my exit code into it's own function.

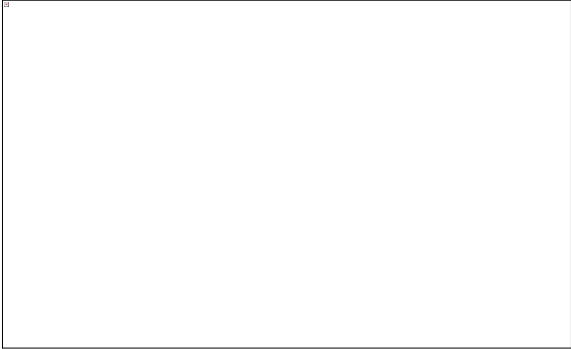
Additionally, the use of iteration and selection were key to making this sprint run. Furthermore, I used file handling (to open and save to the export files) and error handling (to catch the resizing error when it runs out of frame) to help ensure my code works and runs smoothly. As well as using a vast range of different variables and data structures I also used event driven programming to see if an early exit was required.

Testing



As you can see the code has correctly identified the ball and the stumps and even painted over the original frame in blue and green to indicate this.

Evaluation



This section achieves all it set out to. It does take in video, split into frames & identify + export the location of the stumps. However, it’s versability is limited because of the use of the BGR colour scheme. The BGR and RGB colour schemes are very sensitive to changes in light. This means that if the system needs to be moved to another area or played at a different time then the bounds need to be reset. If I were to rebuild this sprint, then I would use the HSV/HLS colour schemes because they’re less sensitive to changes in light/conditions.

I could not use HSV for the sprint because It became extremely difficult to pick out the colour of the ball from the background in the side on view (see above). This meant that I had to use both BGR and HSV to detect the stumps and ball. However, when I tried to do this my computer died and the program ran too slowly to be effective (as evidenced above).

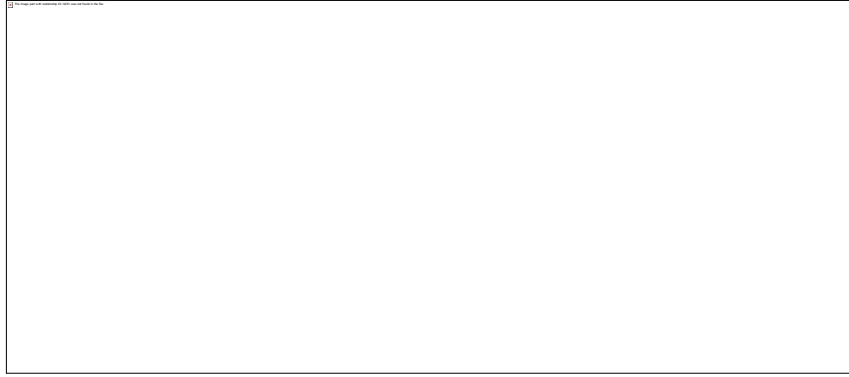
Sprint 6 (Extracting Real world co-ordinates of ball)

According to my plan this sprint needs to:

1	Identify and mark out point of reference
2	Determine ideal camera locations for said point of reference
3	Work out Pixels per Metre
4	Return X,Y & Z co-ordinates

In order to predict where the ball will go, we need to know where the ball has gone and at what rate. Now, we only know the co-ordinates in pixels which isn’t very useful for our calculations. This sprint aims to calculate the 3D co-ordinates of the ball in cm. It does this by finding the height of the stumps in pixels (as these are of known height and are easily identifiable). It then works out how much pixels are in a cm. It then combines this information with co-ordinates from both the cameras to calculate the relative 3D co-ordinates.

Template Algorithm (Pseudocode) for relative co-ordinate system



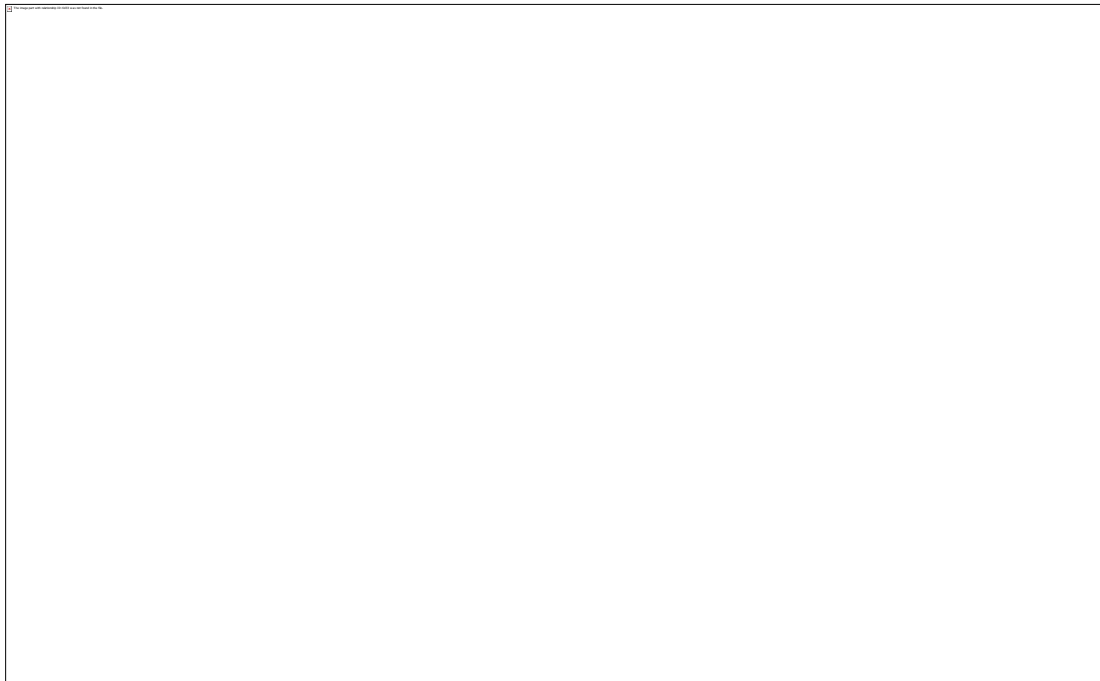
Please Note that this code assumes the cameras are placed in the perfect place so that the corner of the camera view is the corner of the pitch. This means that we don't need to do path correction. This may need to be done but we will learn this with testing.

Initialisation Code



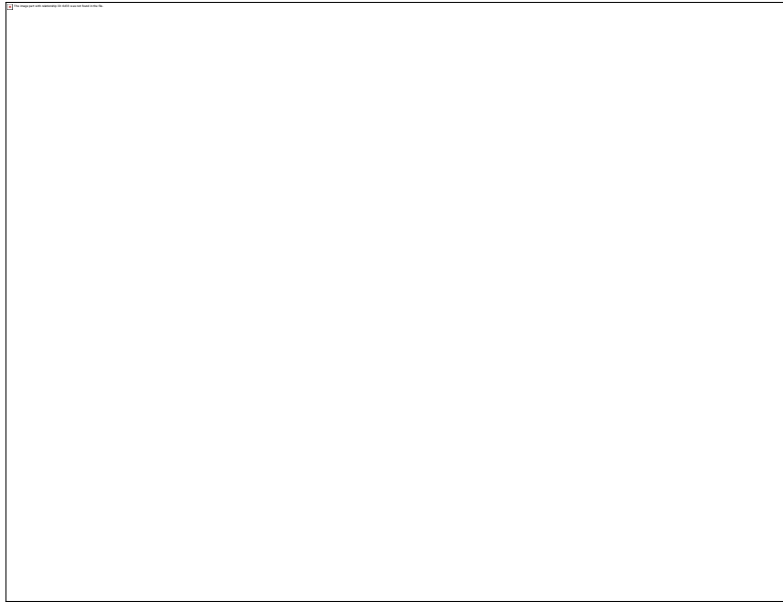
To initialise this sprint, I have to import Regular Expressions (re), copy (for copying creating copies of variables/data structures) and NumPy (for mathematical operations). I also open all the text files where I exported the balls co-ordinates into from the previous sprint.

GetCoordinates()



The function takes in text file of the detected positions of the object detected and returns the co-ordinates. It does this by first iterating through every position and sanitising it using Regular Expressions to remove extra characters exported by the Object Detection sprint. It then places each contour into its own list that contains sub lists of the bounding co-ordinates. Due to the weird formatting exported by OpenCV and NumPy sometimes the co-ordinates will have a empty dimension when they are split by my code. To counteract this I simply removed all blank cells in the co-ordinate list if it's length was not the expected amount of dimensions (2).

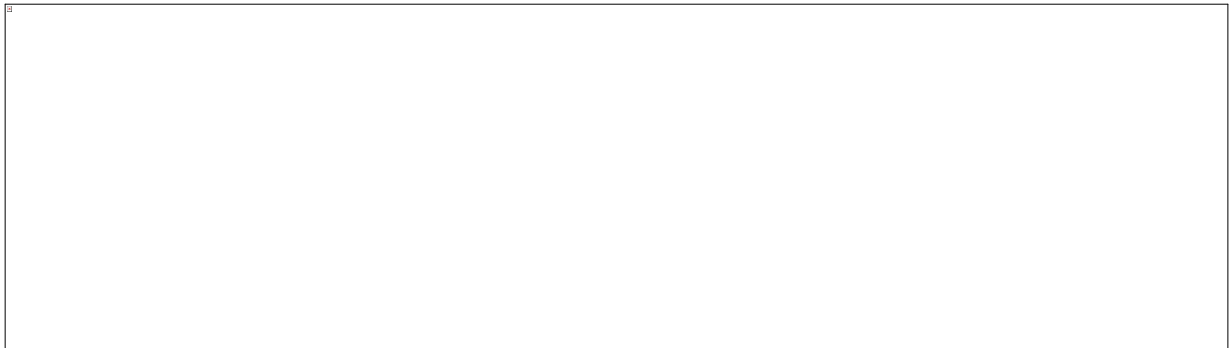
RemoveOutliers()



When I was writing the previous function, I discovered that my Object Detection code was still picking up odd pixels and contours from places where the ball wasn't despite the denoising. This led me to develop this function to remove outliers. I used the traditional statistical method of $1.5 * (Q_3 - Q_1)$ to determine outliers. I decided not to use the $mean \pm 2\sigma$ (*standard deviations*) method because of its high sensitivity to extreme outliers (of which a few of mine were).

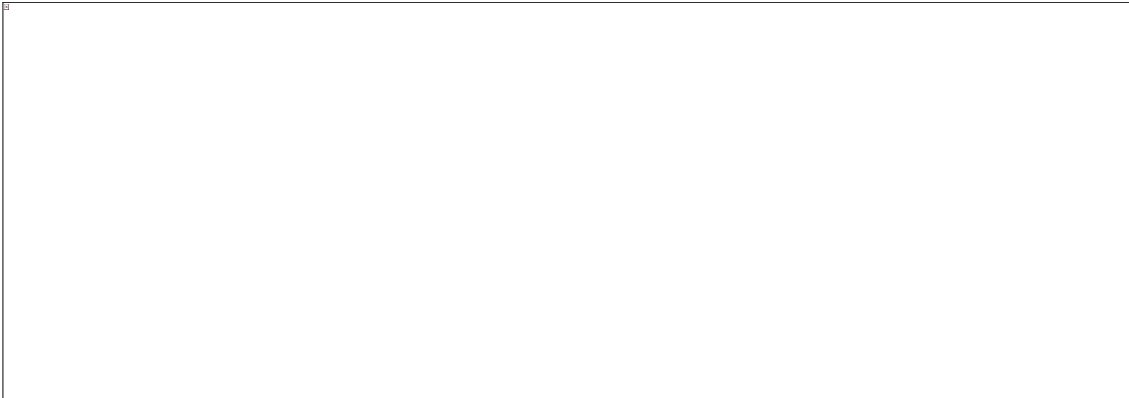
To do this I first sorted the list (that was provided as a parameter to the function) so I could use NumPy to calculate Q_1 & Q_3 . I then calculated the upper bound by adding $1.5 * (IQR)$ to Q_3 . I then did $Q_1 - 1.5 * (IQR)$ to work out the lower bound. Then I just iterated through every element in the list and removed it if it fell outside these bounds.

StumpsHeight()



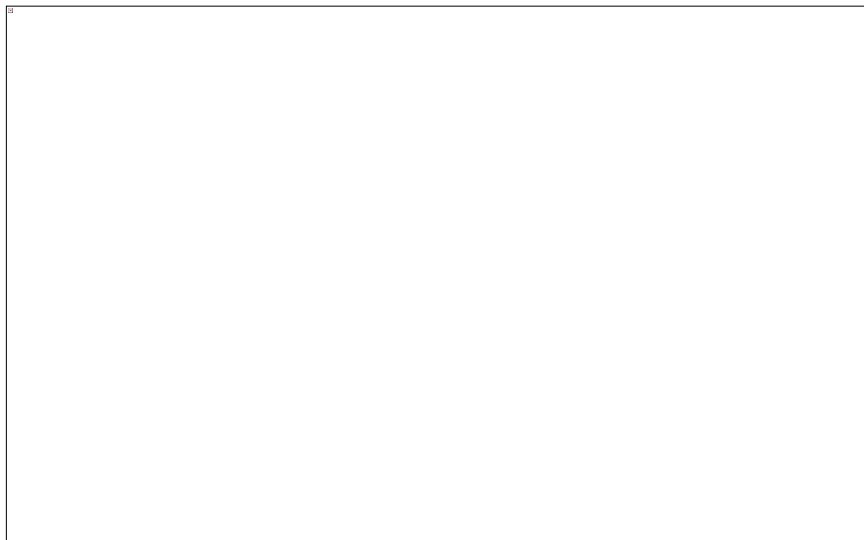
The purpose of this function is to determine the height of the stumps in pixels for the frame provided. It then returns this value. As the stumps are 3 separate sticks instead of just 1 block it's likely that there will be multiple correct contours for the stumps. This means we must iterate through the entire list. I first will remove the contour if it only has one point because then we can't calculate the height. After this, I iterate through the remaining contours and append the difference between the values for y to a list. I append the absolute value to avoid any errors from subtracting in the wrong order and receiving a negative height.

FinalStumpsHeight()



The goal of this function is to calculate the average detected height of the stumps across all frames. To achieve this it iterates through every frame exported and runs the *StumpHeight()* function. It extends the *stump_heights* list with the returned values. It then removes all outliers using the *RemoveOutliers()* function. It returns the mean of *stump_heights* list after all of the outliers have been removed.

BallCoordinates()

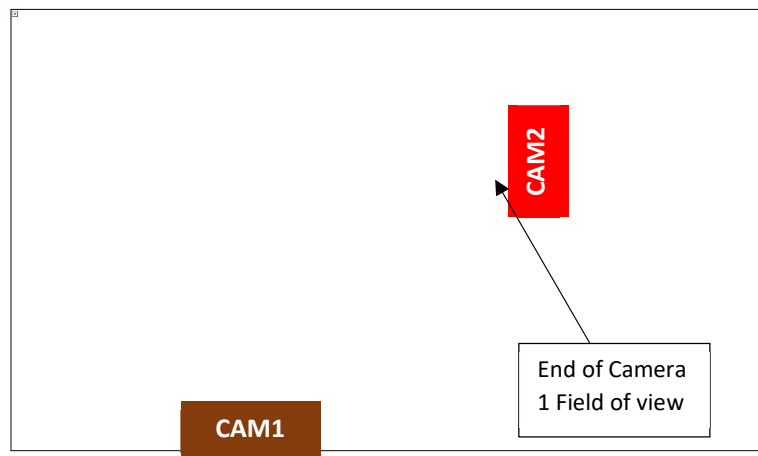


The purpose of this function is to return the relative position of the ball on the pitch. After declaring the *relative_coordinates* list it iterates through every frame and gets the co-ordinates of all the contours. It then creates another two lists to store the x and y values of the vertices detected. It then detects if the contours are empty and if not, it will add their values to the two arrays. After this iterative cycle has finished, it removes all outliers from the two lists. Then it calculates the mean of the two respective lists and multiplies this by *pixelspercm* (a parameter of the function) to figure out the relative position. It then appends the calculated x and y values into the *relative_coordinates* list. Once all the iterative cycles have finished it returns the *relative_coordinates* list.

Launching the functions

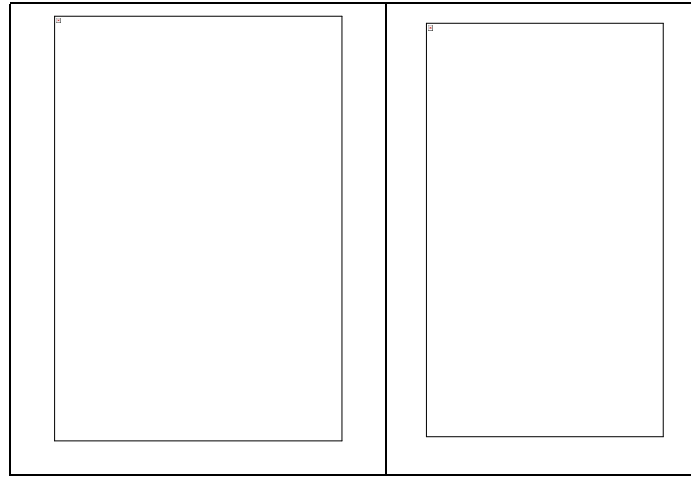


This code simply starts all the functions and records the results in the *coordinates* list. It first figures out the *pixelspercm* of the front camera by calculating the height of the stumps in pixels and then dividing this by 71.12 (the height of stumps in cm). It then gets the relative co-ordinates of the ball from the front camera. It then repeats the same process for the side camera before finally iterating through the co-ordinates and combining the front and side co-ordinates to give 3D relative co-ordinates.



Where to put the camera:

Testing



Although, the frame rates of the two gifs above may be out of sync you should still be able to see the produced positions of the ball accurately follow the position of the ball. I made the graphic on the left by copying the exported ball co-ordinates from the front camera into a free graphing software called DESMOS. Then I made the graphing software highlight the path of the ball with a purple marker to make its path clearer.

Through manual measurements from the video, I determined that my software is roughly accurate to $\pm 0.34cm$.

Variables & Data Structures

Name	Type	Purpose
ball_import	File(read only)	Imported values for position of ball (From Front Camera)
side_ball_import	File(read only)	Imported values for position of ball (From Side Camera)
stump_import	File(read only)	Imported values for position of stumps (From Front Camera)
side_stump_import	File(read only)	Imported values for position of stumps (From Side Camera)
positions	List	List of detected positions
coordinates	List	List of co-ordinates
cleaned	List	String stripped of extra characters and split into list of co-ordinates for use
cleaned_positions	List	List of cleaned co-ordinates
group	List	Group of co-ordinates that make a contour
xy	List	Two long list of co-ordinates (One x co-ordinate and one y co-ordinate)
list	List	List full of values with outliers
q3	Float	3 rd Quartile of list
q1	Float	1 st Quartile of list
upper_bound	Float	Highest possible value w/out being an outlier
lower_bound	Float	Lowest possible value w/out being an outlier

element	Float	The element in the list being compared to the upper and lower bounds
contours_with_area	List	A list full of only contours that have a significant area (no single point/small contours)
values	List	Imported Values from textfile
value	String	Imported values for single frame(Has been split from a text file)
coordinates	List	List of co-ordinates
stump_heights	List	List of recorded values for height of stumps
frame	List	Contours found in the frame of the video
pixelspercm	Float	The amount of pixels in a real world cm
x_vertices	List	List of all the x vertices of a contour
y_vertices	List	List of all the y vertices of a contour
contour	List	Co-ordinates of a contour
contours	List	List of contours
relative_coordinates	List	Real life co-ordinates (in cm)
Front	List	Co-ordinates of ball from front cam
Side	List	Co-ordinates of ball from side cam

Evaluation

This sprint was a lot more complicated than I originally planned however due to my decomposition skills I was able to manage the scale of the sprint and finish it to be fully functional. Calculating the height of stumps was a lot more complicated than I originally anticipated. This is because of the amounts of false and incomplete matches my detection found of the stumps. To account for this I ended up taking an average from every frame and removing any outliers. I believe this is what led my code to achieve a $\pm 0.34cm$ level of accuracy.

However, despite all of the complications my code preformed all I set out for it to do (determining where to put the camera, determining the pixelspercm and returning a 3D relative co-ordinate). I fell overall this sprint has been a success but if I were to redo it I would certainly plan more.

Sprint 7 (Assembling everything together)

Design

Sprint Needs To:

1	Take inputs from functions and programs (created in previous sprints)
2	Determine/ask input for pitching and impact location
3	Plug in bounds of stumps to see if there's a collision. Return results

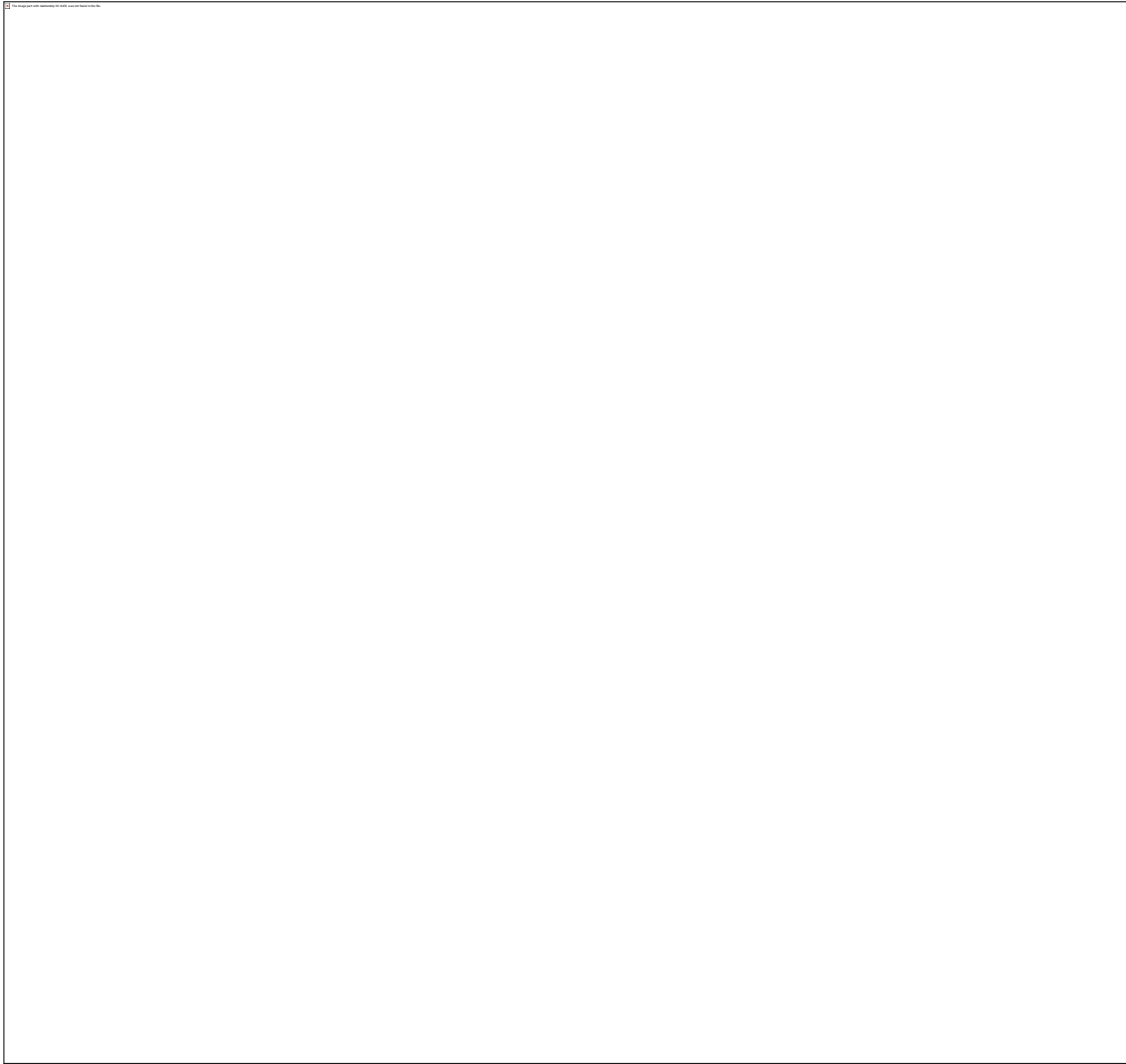
I plan to use all my previous sprints as libraries and import them to one file. Then I will use functional programming to execute these in the right order and pass the data along before finally receiving the final co-ordinates of the ball to see if it is out or not.

Function Name	Purpose	Inputs(s)	Output
SPRINT 1	SPRINT 1	SPRINT 1	SPRINT 1
Ultraedge	AV Processing for Ultraedge function.NB:This isn't a function but rather a batch file	front.mp4, side_imput.mp4, ultraedge.png	ultraedge.avi
SPRINT 2	SPRINT 2	SPRINT 2	SPRINT 2
GetCoefficients()	Launch code for all of the functions required to inverse a matrix.	Input(all x values in matrix), Yvalues(apropiatre y values for x values)	coefficients
Transpose()	Transposes the matrix	Matrix	Transposed
MatrixOfMinors()	Finds the matrix of minors (By iterating through each element and finding the minor)	Matrix	MinorMatrix
Minor()	Find the minor of a element in a matrix	Row, Column, Matrix	Determinant(Matrix)
Detrerminant()	Finds the determinant of a 2x2 matrix	Matrix	ad - bc (Determinant of a 2x2 matrix)
Cofactors()	Finds the matrix of cofactors	Matrix	Matrix - Matrix of cofactors applied
Coefficient()	Returns 1/determinant of the full 3x3 matrix	Matrix	1/det
ScalarMultiply()	Multiplies out a matrix by a scalar factor	Matrix, coefficient (by which to multiply)	MultipliedMatrix
DotProduct()	Returns the dot product of two matrices	Matrix, YMatrix (The two matrices to multiply)	dotproduct - (The final result of the dotproduct)
SPRINT 3	SPRINT 3	SPRINT 3	SPRINT 3
Find1DVelocity()	Finds simple singular dimensional velocity (speed)	coordinates (cleaned co-ordinates of ball in 1 dimension)	(displacement/time) - velocity
SecondRoot()	Finds the second root of the quadratic	initial_vertical_velocity, horrizontal_velocity, e, root1_x	root2_x
Coefficients()	Find the coefficient of the new equation	max_height, root1, root2	[a,b,c] - coefficients of the new equation
GetHeight()	Returns final height of the ball	coefficients	return (coefficients[0]* 500^2) + (coefficients[1] * 500) + coefficients[2] - Final height of the ball
SPRINT 4	SPRINT 4	SPRINT 4	SPRINT 4

PredictLateralMovement()	Predicts how much the ball will move horizontally (Using linear model)	x_coordinates, y_coordinates	(200 - model.intercept_)/model.coef_) - Final Horizontal position
SPRINT 5	SPRINT 5	SPRINT 5	SPRINT 5
ExitTracking()	Exits the tracking loops and saves all recorded positions	N/A	N/A
Main Loop	Finds, highlights, and saves position of ball and stumps. N.B: Not an actual function	ball_location_data.txt, side_ball_location_data.txt, stump_location_data.txt, side_stump_location_data.txt	ball_location_data.txt, side_ball_location_data.txt, stump_location_data.txt, side_stump_location_data.txt
SPRINT 6	SPRINT 6	SPRINT 6	SPRINT 6
RelativeCoordinates	Launches all the functions to find relative positions of the ball. N.B:Not an actual function	N/A	Positions (Uncleaned contour coordinates)
GetCoordinates()	Returns cleaned values for co-ordinates (in pixels)	Positions (Uncleaned contour coordinates)	coordinates (cleaned co-ordinates of contour)
RemoveOutliers()	Removes the outliers from a list (using $1.5 * IQR$)	List (of floats)	List (of floats w/o outliers)
StumpsHeight()	Returns height of stumps in one frame	coordinates (cleaned co-ordinates of contour)	height_values (in pixels)
FinalStumpsHeight()	Returns overall mean height of the stumps in pixels	values(all contour co-ordinates for the stumps)	np.mean(stump_heights) - Mean of the stump_heights list
BallCoordinates()	returns relative co-ordinates of the ball	frame,pixelspercm	relative_coordinates (Of ball in cm)

Pseudocode

I started by writing the pseudocode to make the launch code sections that aren't functions into functions. I.E: add a launch code segment to *Sprint 2, Sprint 3, Sprint 5, Sprint 6*. As these are quite simple, I have bunched these all together.



Unfortunately, I ran out of time here and had to leave working on my project here

Testing & Evaluations

Success Criteria

Here is a quick reminder of what my success criteria looks like:

#Ref	Criteria	How it improves the project/engagement?
1	Processing Time	
1.1	Be able to set up all cameras and microphones in less then 10-15 mins	Will ensure that the users don't spend too long setting it up to prevent them getting frustrated and not using it

1.2	Be able to quickly move all electrical equipment in less than a minute (with 2 ppl).	Will prevent the electrical items from breaking from water damage if it starts to rain quickly. At least 2 ppl can be used (batsman and bowler)
1.3	Process a LBW review with edge detection first in under two mins	Ensures that there is not a large break in play after every review. This time can simply be used to get a drink or for continued practice till the result comes back.
2	Have a reasonable degree of accuracy	
2.1	Be accurate to +- 1.8cm in LBW appeals	Will ensure that people continue to use it. If it was less accurate then this (half the balls diameter) then people would not use it.
2.2	Be able to detect edges	This was one of the most contentious form of dismissals so it's crucial to be able to detect them.
3	GUI	
3.1	Manage and record the amount of LBW reviews done by each time	Prevents players from reviewing any slight chance that will waste peoples time and eventually cause people to stop using it
3.2	Have GUI indicators to show if the cameras, mic and server are all connected	Will allow for quicker and easier set up as well as diagnostics.
3.3	Have buttons to allow the user to be able to select the correct from of review	Allow people to easily review dismissals using the right mode to see if people are out.
3.4	Display the waveform and slow-mo videos for edge detection	Help umpires/ the players decide if there is an edge very easily and quickly.
3.5	Have a setting menu for things like: Ball colour, Stump colour	Will allow the user to use the app on multiple different pitches, with different balls
4	LBW	
4.1	Identify the ball and stumps in a video automatically	Will speed up the time taken per review to prevent the user from getting frustrated and limits manual inputs.
4.2	Detect the path of the ball	The previous path of the ball is instrumental in

		determining if the ball will go on to hit the stumps or not in a LBW appeal
4.3	Predict the height of the ball	Important in deciding if the ball will hit the stumps or not as it will tell us if it goes over the stumps
4.3.1	<i>Calculate the vertical velocity of the ball</i>	Needed in conjunction with the Coefficient of Restitution to predict the height if its' going to bounce
4.3.2	<i>Predict the Coefficient of Restitution</i>	Needed in conjunction with the initial velocity to predict the height if its' going to bounce
4.4	Predict the horizontal movement of the ball	Needed to determine if the ball will be inline with the stumps or not.

Testing the code against Success Criteria

1 Processing Time

1.1 Be able to set up all hardware in 10-15 mins or less

I set up the hardware and pressed the record button 3 times and recorded the times it took me to do this. The first attempt took: 5 minutes 53 seconds, the next attempt took 6 minutes and 5 seconds, and the final attempt took 5 minutes 28 seconds. On average this is: 5 minutes 49 seconds. This is well under the maximum time I stated in my report. However, it is worth noting that as my project is not finished this could be longer in the future. Things, like syncing the devices and starting the server might take longer. However, I don't believe this will take a significant enough amount of time to make it fail this test.

✓ Test Passed

1.2 Be able to quickly move all electrical equipment in less than a minute (with 2 ppl).

I ran this test three times with my brother. In order to maximise efficiency, I informed the batsman to pick up and collect the mic and camera and from the batsman's end. Meanwhile I made the bowler sprint up and take the camera from the side on their way indoors. All three times the test just about passed with times of 57 seconds, 54 seconds and 58 seconds. This would not take longer even if my project was more complete as the amount of hardware or its' location would not change.

✓ Test Passed

1.3 Process a LBW review with edge detection first in under two mins

Much like the other trials I also ran this trial three times. But I ran into difficulties because my project is not fully complete. This meant that I had to manually rip the video feeds of storage of the phones as well as manually run them through the programs and manually copy paste the results across the files. This test was a complete failure because Sprint 7 (Assembling together all the different parts of the backend) and Sprint 10 (linking the frontend to the backend) were not done so everything had to be done manually. The first time 8 minutes 32 seconds whereas the second time took 6 minutes 45 seconds and the final try took just 5 minutes 25 seconds. The times reduced as I got more used to the project but would never get down to less than 1 minute without the automations. If I had more time, I would rectify this by immediately finishing sprint 7 and the GUI.

✗ Test Failed

2 Having a reasonable degree of accuracy?

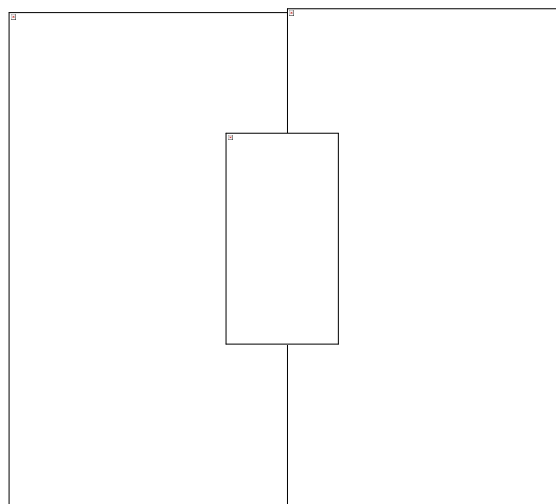
2.1 Be accurate to +/- 1.8cm in LBW appeals

As mentioned in the previous test. Due to my project not being fully finished I couldn't run through reviews completely unless I did this manually. The number of samples I could run and measure in the limited time wouldn't be enough to pass judgement on the accuracy. However, on the tracking of the existing path my program achieved an accuracy of $\pm 0.34\text{cm}$. This leaves a margin of 530% for the tracking to pass. Whilst this extremely likely to pass as I can't test this yet, I can't pass judgement on this criteria.

✓ ✗ Unable to complete test but likely to pass

2.2 Be able to detect a bounce on the edge detection test

As the tests in Sprint 1 detected the software is very capable of determining even the faintest of edges. Here is the copy of the testing from scene 1:



✓ Test Passed

3 GUI

3.1 – 3.3,3.5 GUI Elements

All these tests have failed as I didn't have time to make the GUI. This has made the program suffer immensely as it has made it harder to use, take longer to set up and longer to run. This would be my next immediate priority if I had more time. This would allow my software to be used by the average consumer.

✗ Test Failed

3.4 Display the waveform and slow-mo videos for edge detection

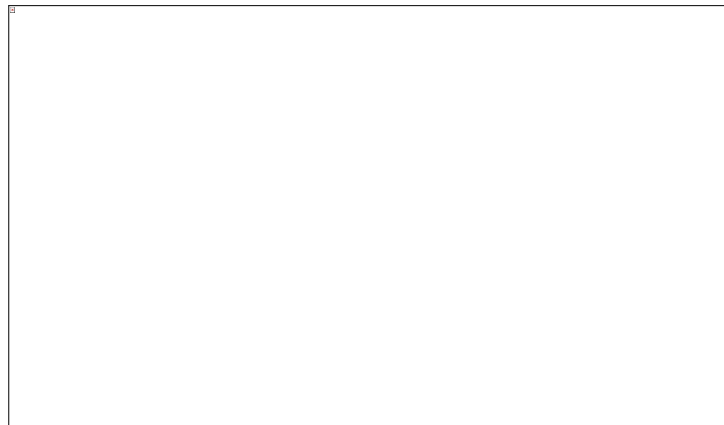
I have managed to meet this criterion in the first sprint. Through using FFMPEG. However, this feature was achieved quite basically and I was planning to have a screen in the GUI to let you run this slow motion video through frame by frame to help determine if the batsman has edged it or not. So whilst this has passed, it has not passed to its' fullest extent.

✓ Test Passed (but could be better)

4 LBW laws

4.1 Identify the ball and stumps in a video automatically

My code can correctly identify the stumps and ball automatically given that the parameters provided for the ball and stumps are correct. However, as Sprint 7 is not yet complete these values are currently hardcoded and must be manually entered into the code for each new set of stumps/balls



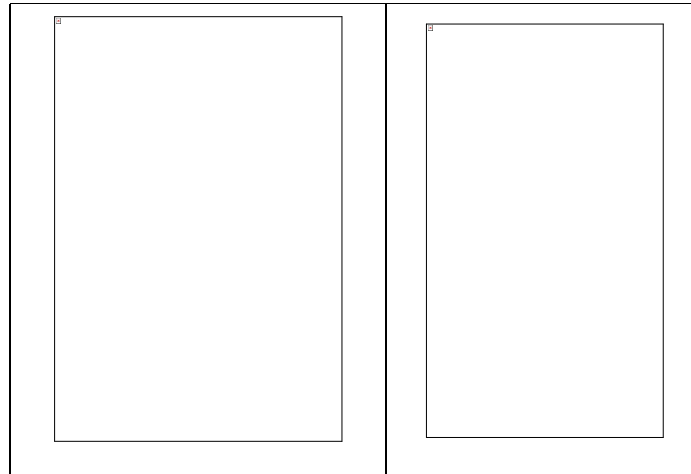
for now. As you can see below the code is clearly able to pick out and highlight the ball and stumps automatically with no further inputs.

Please note that this image was taken from mid-project testing above

✓ Test Passed

4.2 Detect the path of the ball

My code was able to detect the path of the existing path of the ball very accurately by using statistical methods on the data collected by my program to ensure correct co-ordinates were detected in each frame. It then spat out the real-life co-ordinates of the balls position in relation to the pitch. I have plotted the path of the ball in DESMOS show that the path of the ball matches with the video. The gaps between the points increases because the velocity increases and the ball goes further away from the camera.



Please note that this image was taken from mid-project testing above

✓ Test Passed

4.3.1 Calculate the vertical velocity of the ball

My code can calculate the vertical velocity of the ball using iteration and simple mathematical techniques. Unfortunately, there is no easy way to test if the code works without having a RADAR. So I decided to run the code through my test data just to see if the answer it returned was believable. If it was then I would determine that it has passed if not, then it has failed. I received an answer 12.025m/s which is within the bounds of what I would expect.

✓ Test Passed

4.3.2 Predict the Coefficient of Restitution

I didn't Predict the coefficient of restitution since I was running short on time and it would take a long time to train a model to do this. This would make the height prediction inaccurate because lots of factors like wind speed and humidity effect the Coefficient of restitution. If I had more time I would definitely upgrade this to use a K nearest neighbour regression to predict this instead of just using a pre-searched up value on the internet.

✖ Test Failed

4.4 Predict the horizontal movement of the ball

I modelled horizontal movement as linear as the only two forces that can act on the ball are air resistance and the accelerative force of the bowler releasing the ball from an angle. As cricket balls are very heavy air-resistance is largely negligible unless the ball is swinging. I got the answer of 48.0467593 as the final position of the ball and when I compared this to the actual video and measured it manually, I got 48. So this is incredibly accurate and passes the test.

✔ Test Passed

A note on validation testing (valid/invalid/boundary)

As all of the data used in my program is directly imported from camera feeds/videos there is no need to test valid, invalid and boundary inputs as the user can only import a video and nothing else. If the program doesn't find a ball or stumps (invalid case) then it will simply stop and exit as most things are run using *for each* loops.

Survey Questions

I decided to offer out a follow up survey to see what people thought of my program. I built this form the same way in Microsoft Forms.

The first question I asked was "On a scale of 1-5 how usable do you think the program is in its' current state?". I had given them a demo of the software beforehand. I am treating them as my focus group.

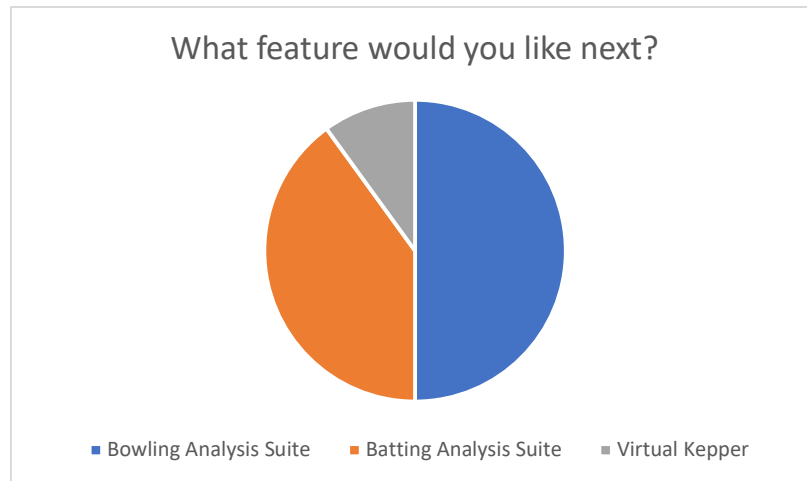
In the next question, I included the picture of the mock-up of my UI found in my design section and asked them, "If you saw this UI in an app would you know how to use it?". This was to test if my app UI was as simple and as self-explanatory as I thought. And to see if I were to have more time if people would have been satisfied with the UI.

The third question I asked was "Which feature would you like to be added next (after the GUI)?" and gave them the options of: "Ball analyser (pitch maps, movement in the air, slow-mo video)", "Batting Analysis suite", "Virtual stumping predictor and crease".

Results of survey

The mean response to the first question was a score 1.07. Everybody but one person responded 1, except one person who responded 2. This was to be expected as there was no GUI and everything had to be done manually. Sprint 7 and GUI were my two next priorities anyway.

Roughly 90% of all respondents said that they would understand what to do in the app without further instruction if it was real. This shows that the app design is actually fairly simple and met all my goals.



The overwhelming majority wanted the Batting and Bowling suites whereas a small minority (roughly 10%) said the Virtual Keeper feature. This confirms the results of the first survey and helps me focus on what I would develop if I had more time after patching everything else up.

If I had more time and what things I would do differently

If I had more time, I would account for the magnus effect. I didn't do as this is a very complicated force with complicated physics that I didn't quite understand when I was planning my project. But now that I have a greater grasp, I believe that I could include it in an update. The magnus effect is the name given to the force that is applied to objects that are spinning in the air. This will cause them curve and dip. If I were to record this, I could also use this in the Bowling Analysis part when that gets made.

I would expand on the GUI further after it's finished by having the predicted position of the ball show up on a 3D animation of the trajectory. This would make it easier for users to visualise the path of the ball and see why it's out rather than just seeing the word "Out!".

I would also move the side camera back more and use a wide-angle lens. I would then correct for distortion and use that to generate the co-ordinates. This would mean I have more data points and therefore more accuracy.

If I were to remake the project again, I would make quite a few changes. One of which would be to make the side camera feed, front camera feed and ultraedge overlay in FFMPEG rather than leaving it to do in the GUI. This is because FFMPEG is done server side so the phone (display and camera) needs to do a lot less processing. This means that it can be even less powerful.

Another change I would make is to use two different colour schemes for the ball and the stumps. (HSV and BGR) this is because the ball was clashing under HSV but the stumps weren't. It's always better to use HSV where possible as it's more resilient to changes in light. I could not do this before

as my computer (server) was not powerful enough to run both simultaneously. However, I now realise that I can parallelise OpenCV using a library called OpenCL that would allow me to be able to run both colour schemes at once.